
The Surprising Difficulty of Search in Model-Based Reinforcement Learning

Wei-Di Chang^{*12} Mikael Henaff¹ Brandon Amos¹ Gregory Dudek² Scott Fujimoto^{*1}

Abstract

This paper investigates search in model-based reinforcement learning (RL). Conventional wisdom holds that long-term predictions and compounding errors are the primary obstacles for model-based RL. We challenge this view, showing that search is not a drop-in replacement for a learned policy. Surprisingly, we find that search can harm performance even when the model is highly accurate. Instead, we show that mitigating overestimation bias matters more than improving model or value function accuracy. Building on this insight, we identify that taking the minimum over an ensemble of value functions effectively addresses this bias and enables effective search, achieving state-of-the-art performance across multiple popular benchmark domains. Code can be found at <https://github.com/facebookresearch/MRSQ>.

1. Introduction

Model-based reinforcement learning (MBRL) has long been regarded as a promising paradigm for sample-efficient decision-making in complex environments. By learning an explicit dynamics model, agents can simulate future trajectories, plan ahead, and make informed decisions without requiring millions of real-world interactions (Sutton, 1991).

Despite its conceptual appeal, MBRL has often struggled to meaningfully outperform its model-free counterparts (Van Hasselt et al., 2019; Schwarzer et al., 2023; Fujimoto et al., 2025). A common explanation for this shortfall is model accuracy and compounding errors (Talvitie, 2014; Venkatraman et al., 2015; Asadi et al., 2018; Lambert et al., 2022). When a model is used to simulate trajectories over long horizons, small prediction errors accumulate, lead-

ing to increasingly unreliable state estimates. This phenomenon has motivated a substantial body of research focused on improving model accuracy (Oh et al., 2015; Nagabandi et al., 2018; Hafner et al., 2019), uncertainty-aware models (Deisenroth & Rasmussen, 2011; Chua et al., 2018; Janner et al., 2019; Henaff et al., 2019), and long-horizon prediction (Oh et al., 2017; Silver et al., 2017; Lambert et al., 2021; Janner et al., 2022; Ma et al., 2024; Farebrother et al., 2025). The implicit assumption underlying these efforts is that better models will directly translate to better planning and, consequently, better performance.

In this work, we challenge this assumption by advancing a core hypothesis: *model accuracy alone does not determine the effectiveness of search*. We explore this hypothesis through the lens of MR.Q (Fujimoto et al., 2025), a state-of-the-art algorithm that learns a dynamics model purely for representation learning, without incorporating search.

We begin in Section 4.1 by establishing a counterintuitive result: search can fail even with a *perfect* dynamics model and value function. Beyond a critical planning horizon, the search space expands dramatically, rendering exhaustive search computationally infeasible and high-value trajectories vanishingly rare under sampling-based approaches. Critically, this limitation is intrinsic to the search process; improving model quality cannot resolve it.

We next ask whether these theoretical limitations appear in practice. Our empirical analysis in Section 4.2 corroborates this theoretical limitation. Despite MR.Q’s architectural similarity to TD-MPC2 (Hansen et al., 2024), a leading search-based MBRL method, we find that naively incorporating search *harms* MR.Q’s performance. This occurs even though MR.Q’s learned dynamics are comparably accurate to those of TD-MPC2, which benefits substantially from search. These results confirm that model accuracy alone does not determine whether search will be effective, but they also raise a natural question: if not model accuracy, what *does* determine the success of search?

We argue that the primary obstacle lies in the interaction between search and value learning. Because exhaustive search is infeasible, practitioners rely on sampling-based methods guided by a learned value function. Due to the computational cost of search, this value function is typically trained using rollouts from a learned, non-search pol-

^{*}Equal contribution ¹Meta FAIR ²McGill University, Montréal, Canada. Correspondence to: Wei-Di Chang <wei-di.chang@mail.mcgill.ca>, Scott Fujimoto <sfujimoto@meta.com>.

icy network (Hansen et al., 2024). In Section 4.3, we show that this creates a mismatch between data collected with search and the distribution assumed during training, leading to overestimation bias that degrades both value function quality and overall performance (Fujimoto et al., 2019).

Rather than trying to build better models, we directly address the overestimation bias introduced by search by using the minimum over an ensemble of value functions to produce pessimistic estimates for out-of-distribution actions. We instantiate this approach in a new algorithm: Model-based Representations for Search and Q-learning (MRS.Q). We evaluate MRS.Q on over 50 tasks spanning multiple benchmark domains using a single fixed set of hyperparameters. Our results demonstrate consistent improvements over both state-of-the-art model-free and model-based methods, validating that addressing overestimation, rather than solely improving model quality, is key to unlocking the benefits of search.

Our findings show the bottleneck is not just the model, but how we use it. Progress lies not only in building better dynamics models, but also in understanding and managing the fundamental tensions between value learning and search.

2. Related Work

MBRL. MBRL approaches can be broadly categorized by how they utilize their model. The simplest approach trains a model-free RL algorithm on synthetic samples generated by the model (Atkeson & Santamaria, 1997; Abbeel et al., 2006; Ha & Schmidhuber, 2018; Janner et al., 2019). A related strategy integrates the learned model into value learning by generating multi-step returns (Oh et al., 2017; Feinberg et al., 2018; Buckman et al., 2018; Hafner et al., 2019; 2023; Amos et al., 2021). The PILCO family of methods (Deisenroth & Rasmussen, 2011; Gal et al., 2016; Higuera et al., 2018) takes a different approach, using gradients from the model to directly improve the policy.

Search in MBRL. A more sophisticated approach uses the world model for search. One common strategy, rooted in optimal control, is model predictive control (MPC), which repeatedly searches through the model to identify the best action at each time step (Draeger et al., 1995; Levine & Koltun, 2013; Mordatch & Todorov, 2014; Watter et al., 2015; Finn et al., 2016; Gu et al., 2016; Ebert et al., 2017; Banijamali et al., 2018; Nagabandi et al., 2018; Chua et al., 2018; Lowrey et al., 2019; Henaff et al., 2019). For discrete action spaces, Monte Carlo tree search offers a related alternative (Schrittwieser et al., 2020; 2021; Ye et al., 2021; Wang et al., 2024). The current state of the art for MPC-based methods is TD-MPC2 (Hansen et al., 2022; 2024), which uses short-horizon search with a value function.

Challenges in MBRL. MBRL faces several fundamental

challenges. Compounding error in auto-regression predictions limits the ability to plan over long horizons (Talvitie, 2014; Venkatraman et al., 2015; Asadi et al., 2018; Lambert et al., 2022). The objective mismatch problem (Lambert et al., 2020): models are trained to optimize immediate predictions but are ultimately evaluated by policy performance. Palenicek et al. (2023) showed that oracle models offer diminishing performance gains. MR.Q (Fujimoto et al., 2025) demonstrates that models can improve representation learning without search, raising the question of when and how search adds value. Our empirical observations reinforce these concerns, suggesting that how the model is used matters more than its accuracy. Recent extensions of TD-MPC2 have also highlighted a performance gap between the learned policy network and MPC, applying regularization to align the policy with MPC-selected actions (Lin et al., 2025; Wang et al., 2025; Zhan et al., 2025). Lin et al. (2025) demonstrate that MPC causes the value function to overestimate the performance of its policy network. Our experiments reveal that this failure mode extends beyond TD-MPC2, and that the value function can also overestimate the online, MPC-based policy.

Overestimation bias. Overestimation bias is a well-known problem in RL, stemming from the maximization of an approximate value function (Thrun & Schwartz, 1993; Van Hasselt, 2010; Van Hasselt et al., 2016; Fujimoto et al., 2018). In offline RL, overestimation bias is caused by errors induced by the distribution shift between the offline dataset and the learned policy (Fujimoto et al., 2019). A similar dynamic arises when search selects actions outside the distribution seen during value function training (Lin et al., 2025; Wang et al., 2025; Zhan et al., 2025). A common strategy for addressing overestimation bias is through the minimum of value functions (Fujimoto et al., 2018; Lan et al., 2020; An et al., 2021). In our work, we show that this technique is particularly effective for mitigating the overestimation induced by search.

3. Background

Reinforcement learning (RL). We follow the standard formulation (Sutton & Barto, 1998), where an agent interacts with an environment modeled as a Markov Decision Process (MDP) defined by the tuple (S, A, p, R, γ) , with state space S , action space A , dynamics function p , reward function R , and discount factor γ . The objective is to learn a policy π mapping state $s \in S$ to action $a \in A$, that maximizes the expected discounted return $\mathbb{E}_\pi [\sum_{t=0}^{\infty} \gamma^t r_t]$ of rewards $r_t \sim R(s_t, a_t)$. The value function $Q^\pi(s, a) = \mathbb{E}_\pi [\sum_{t=0}^{\infty} \gamma^t r_t | s_0 = s, a_0 = a]$ measures the expected return when starting in state s and taking action a .

Model-based reinforcement learning (MBRL) approaches learn an explicit dynamics model of the environment that

predicts the next state s_{t+1} given the current state s_t and action a_t . In this paper, we focus on methods that use these learned models to plan via search. Search refers to the process of evaluating candidate action sequences through model rollouts to select actions. Model predictive control (MPC) is an online search framework that optimizes actions over a finite horizon using model rollouts, executing only the first action before re-planning at each time step.

TD-MPC2. TD-MPC2 (Hansen et al., 2024) is a MBRL algorithm that learns a latent world model that is used for MPC. The state embedding $\mathbf{z}_s$ is learned together with the dynamics function, reward model, and value function by co-optimizing loss functions for each:

$$\begin{aligned} \mathcal{L}(\mathbf{z}_s, F, R, Q) = & \mathcal{L}_{\text{Dynamics}}(F(\mathbf{z}_s, a) - \mathbf{z}_{s'}) \\ & + \mathcal{L}_{\text{Reward}}(R(\mathbf{z}_s, a) - r) \\ & + \mathcal{L}_{\text{Value}}(Q(\mathbf{z}_s, a) - (r + \gamma Q(\mathbf{z}_{s'}, \pi(\mathbf{z}_{s'})))) \end{aligned} \quad (1)$$

During MPC, trajectories $\tau = (\tilde{\mathbf{z}}_0, a_0, \tilde{\mathbf{z}}_1, a_1, \dots, \tilde{\mathbf{z}}_N)$ are generated by recursively rolling out the model from an embedding $\mathbf{z}_{s_0}$ of an initial observed state s_0 :

$$\tilde{\mathbf{z}}_t = F(\tilde{\mathbf{z}}_{t-1}, a_{t-1}) \quad \text{where} \quad \tilde{\mathbf{z}}_0 = \mathbf{z}_{s_0}. \quad (2)$$

The value V of a trajectory τ is computed by summing a final value Q with predicted rewards along the model-predicted embeddings over a finite horizon N :

$$V(\tau) = \sum_{t=0}^{N-1} \gamma^t R(\tilde{\mathbf{z}}_t, a_t) + \gamma^N Q(\tilde{\mathbf{z}}_N, \pi(\tilde{\mathbf{z}}_N)). \quad (3)$$

TD-MPC2 uses a learned policy network π , trained to maximize value, in its value updates, but selects actions during environment interaction using Model Predictive Path Integral (MPPI) control (Williams et al., 2015). At each time step, candidate action trajectories are initially sampled from a mixture of random and policy-derived actions, then evaluated following Equation 3. The highest-valued trajectories inform the next round of candidate sampling, and this process repeats. The final trajectory is selected through value-weighted sampling, and its first action is executed.

MR.Q. MR.Q (Fujimoto et al., 2025) is a model-free RL method that uses a model-based objective to learn a state-action embedding $\mathbf{z}_{sa}$ which has an approximately linear relationship to the value function $Q^\pi(s, a)$. Unlike most model-based methods, which use search or model rollouts, MR.Q’s objective is solely used to learn useful representations. More formally, MR.Q’s learning objective co-optimizes the embedding $\mathbf{z}_{sa}$ with weights capturing the reward and dynamics of the environment:

$$\begin{aligned} \mathcal{L}(\mathbf{z}_s, W_p, W_r) = & \mathcal{L}_{\text{Dynamics}}(\mathbf{z}_{sa}^\top W_p - \mathbf{z}_{s'}) \\ & + \mathcal{L}_{\text{Reward}}(\mathbf{z}_{sa}^\top W_r - r). \end{aligned} \quad (4)$$

The value function is trained independently with a standard Bellman update (Mnih et al., 2015; Lillicrap et al., 2015), taking the minimum between two value functions in the target (Fujimoto et al., 2018):

$$\mathcal{L}_{\text{Value}}(r + \gamma \min(Q_1, Q_2)(\mathbf{z}_{s'a'}) - Q(\mathbf{z}_{sa})), \quad (5)$$

where the action a' is sampled from the policy $\pi(\mathbf{z}_{s'})$. The state embedding $\mathbf{z}_s$ is learned upstream, end-to-end with $\mathbf{z}_{sa}$, i.e., $g(\mathbf{z}_s, a) = \mathbf{z}_{sa}$. Since $\mathbf{z}_{sa}$ is a non-linear function of $\mathbf{z}_s$ and a , the resulting dynamics and reward function are functionally similar to TD-MPC2’s. Crucially, although MR.Q leverages a model-based objective, it is strictly used to learn the embeddings rather than forward dynamics predictions or search.

Adding and removing MPC. The architectural similarity between TD-MPC2 and MR.Q allows us to isolate the effect of search by adding MPC to MR.Q and removing it from TD-MPC2. For clarity, let π_{MPC} denote the implicit policy induced by MPC, which selects actions via MPPI using the learned model and value function (Equation 3). Methods with MPC select actions according to π_{MPC} , while methods without MPC select actions directly from the learned policy π .

Distribution shift. Distribution shift arises in value estimation when the behavior policy used for data collection differs from the target policy π used in the value update (Equations (1) and (5)). For example, if MPC selects actions during data collection but the value function is trained with actions from the learned policy π , this mismatch may cause the value function to be queried on actions it was not trained on, ultimately leading to unreliable value estimates (Fujimoto et al., 2019).

4. The Challenge of Search

In this section, we investigate the fundamental challenges limiting the effectiveness of search in MBRL. Conventional knowledge suggests that model accuracy is the key predictor of MBRL performance. We challenge this assumption, demonstrating that it does not strictly hold. While model accuracy remains important, our results suggest that there are other underappreciated factors at play.

We present three key findings:

1. **Naive search can fail with a perfect model.** We show theoretically that even with perfect model and value functions, uniform random search can fail with high probability over longer horizons due to the exponential growth of the search space (Section 4.1).
2. **Model accuracy alone does not determine whether search improves performance.** Contrary to expectations, adding MPC to MR.Q (Fujimoto et al., 2025) degrades its performance, despite MR.Q having a more

accurate model than TD-MPC2 (Hansen et al., 2024), a method where MPC proves beneficial (Section 4.2).

3. **Search induces overestimation bias.** We show that simply replacing the learned policy with MPC when collecting data introduces overestimation bias that degrades value function quality (Section 4.3).

Together, these findings motivate the need for methods that explicitly address these challenges rather than focusing solely on improving model accuracy.

4.1. Search with a perfect model

We begin by examining the theoretical limits of search under ideal conditions. Perhaps surprisingly, search can fail with high probability even with perfect dynamics and a perfect value function. To illustrate this, consider the N-chain MDP defined in Figure 1. In this environment, a single action a_0 advances the agent along the chain, while all other actions lead to an absorbing state. The reward is zero everywhere except at the final state. We define search failure as the inability to discover any trajectory with non-zero value, and analyze a uniform random search that samples action sequences, simulates trajectories via the model, and selects the trajectory with the highest value.

In this N-chain environment, regular states have zero reward but non-zero value, since the final state remains accessible. In contrast, the absorbing state has both zero reward and zero value. Consequently, if the agent selects any action other than a_0 , the trajectory will end in the absorbing state and will necessarily have zero value. Therefore, even with access to the ground-truth value function, an agent can only select action a_0 at each time step to discover a trajectory with non-zero value.

Under random search, the probability of finding such a trajectory is $1 - (1 - \frac{1}{A^n})^m$, where A is the number of actions, n is the search horizon, and m is the number of sampled trajectories. Intuitively, only 1 of A^n possible action sequences consists entirely of a_0 , so the probability of randomly sampling a successful trajectory decreases exponentially with horizon length.

In Figure 2 we show that this probability degrades rapidly as the search horizon increases. For example, with $A = 10$ actions and $m = 1000$ trajectories, a search horizon of $n = 3$ gives a probability of approximately 0.63 of discovering a non-zero valued trajectory. However, extending the search horizon to $n = 10$ causes the probability to fall to 10^{-7} .

The failure in this example could be avoided by acting according to the value function rather than relying on search. More broadly, this illustrates that depending solely on imagined trajectories may be insufficient when the search space is large, particularly in domains with continuous

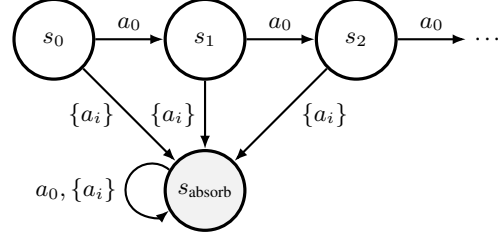


Figure 1. **Absorbing N-chain.** The absorbing N-chain is a simple environment with $N + 1$ states and A actions. The action a_0 moves the agent forward and any other action $\{a_i\}$ sends the agent to a self-looping absorbing state s_{absorb} . The reward is 0 everywhere, other than the final state s_{N-1} of the N-chain.

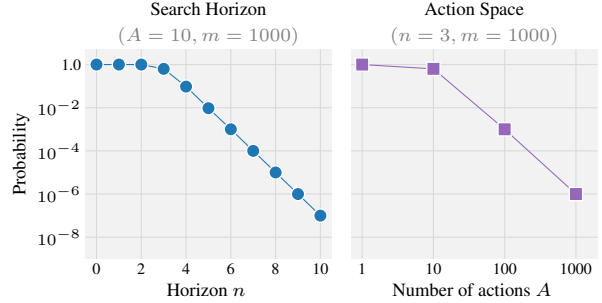


Figure 2. **Probability of finding a non-zero value trajectory.** We analyze how the probability of discovering a non-zero value trajectory varies with the search horizon length (left) and the action space size (right) in the absorbing N-chain (Figure 1) according to $1 - (1 - \frac{1}{A^n})^m$. This demonstrates that the probability decays rapidly as either parameter increases.

states and actions. While modern search-based methods such as TD-MPC2 (Hansen et al., 2024) use policy-guided sampling to narrow the search space, they still face combinatorial challenges as the horizon increases, particularly when the policy is imperfect or the reward landscape requires precise action sequences. This helps explain why MBRL methods often employ short search horizons (three time steps), both to constrain the search space and to reduce compounding model error.

4.2. Does model accuracy determine performance?

Having established that search can fail even with perfect models in theoretical settings, we now examine the role of model accuracy in practical settings with learned components. We compare two recent state-of-the-art methods, MR.Q (Fujimoto et al., 2025) and TD-MPC2 (Hansen et al., 2024), both with and without MPC. We focus on these methods because the similarities in architecture and learning objectives allow us to isolate the effect of search from confounding factors.

In Table 1 we evaluate each method’s learned dynamics model accuracy and task performance using the following metrics:

- **Dynamics error.** The mean-squared error between predicted and actual next-state embeddings, weighted by γ^t over the three-step search horizon. To ensure comparable error magnitudes, we modify MR.Q’s state encoder by replacing its final activation function with Simplicial Embeddings (SEM) (Lavoie et al., 2023), matching TD-MPC2’s architecture and ensuring that differences in embedding scale do not inflate one method’s error.
- **Unroll error.** The absolute error between the value prediction used in MPC and the ground-truth trajectory value. We compute the value prediction by unrolling the dynamics over the search horizon, and summing the discounted predicted rewards with the value estimate of the final predicted state (Equation 3). The ground-truth value is computed similarly, unrolling trajectories with the simulator and summing the true discounted reward over the full episode length.
- **Performance Δ .** The performance difference with and without MPC (i.e., MR.Q with MPC vs. MR.Q, and TD-MPC2 vs. TD-MPC2 without MPC). Positive values indicate that MPC improved performance.

Dynamics and unroll errors directly measure model quality. Unroll error, in particular, measures the quality of the signal that MPC uses to select actions (Equation 3), as it reflects both dynamics and value prediction accuracy. Performance Δ shows the change in performance using the *identical* MPC procedure with each algorithm.

Despite comparable model accuracy between methods, using MPC gives divergent outcomes, improving TD-MPC2 while degrading MR.Q. Table 1 quantifies this surprising asymmetry. Although both methods exhibit similar dynamics and value prediction errors, TD-MPC2’s performance improves substantially with MPC across nearly all DM Control Suite (DMC) tasks (Tassa et al., 2018), while MR.Q’s consistently decreases. Given their architectural and loss function similarities, we would expect comparable error rates to translate into comparable search benefits.

The substantial gap in MPC performance confirms that model accuracy alone does not determine whether search will help, suggesting other factors are at play.

4.3. Why does search harm MR.Q?

Given that model accuracy does not fully explain these results, we now investigate what other factors are responsible. Following Lin et al. (2025); Wang et al. (2025); Zhan et al. (2025), we hypothesize that distribution shift and its downstream effects on value estimation are the key bottleneck to search performance.

Consider the standard value function update used by both TD-MPC2 and MR.Q, based on a value target that evalu-

Table 1. Comparing model accuracy and performance. Dynamics error and unroll error do not correlate with MPC performance. For dynamics error and unroll error, we highlight which method performs better or worse. Performance Δ indicates whether MPC improved or harmed each method’s performance.

Environment	Dyn. Error	Unroll Error	Perf. Δ	
MRQ	acrobot-swingup	3.62e-07	2.98	-107.41
	cheetah-run	3.16e-05	2.26	-173.53
	dog-stand	8.59e-05	15.53	-238.35
	dog-walk	7.36e-05	11.13	-260.71
	dog-trot	2.56e-05	3.32	-207.10
	dog-run	3.51e-05	1.14	-208.12
	hopper-stand	2.48e-05	3.09	-27.87
	hopper-hop	3.77e-05	14.11	-26.83
	humanoid-stand	5.30e-05	3.98	-756.61
	humanoid-walk	1.22e-04	14.93	-291.00
	humanoid-run	4.18e-05	1.87	-95.71
	walker-run	3.04e-05	12.51	-263.36
	(Gym) Ant	4.78e-05	136.90	166.23
	(Gym) HalfCheetah	5.39e-05	207.99	-8395.10
	(Gym) Hopper	3.63e-05	12.81	-1990.26
	(Gym) Humanoid	5.23e-05	809.28	-5692.61
	(Gym) Walker2d	2.47e-05	36.60	-5869.88
TD-MPC2	acrobot-swingup	1.39e-06	7.15	121.01
	cheetah-run	4.85e-05	3.37	40.04
	dog-stand	4.36e-05	3.68	-348.53
	dog-walk	1.00e-04	2.67	2.71
	dog-trot	8.21e-05	69.30	80.41
	dog-run	1.25e-04	5.33	23.25
	hopper-stand	2.12e-04	33.89	236.21
	hopper-hop	1.61e-04	6.72	202.32
	humanoid-stand	3.37e-05	41.22	234.44
	humanoid-walk	8.83e-05	40.62	273.34
	humanoid-run	9.33e-05	9.77	66.86
	walker-run	4.42e-05	12.21	157.90
	(Gym) Ant	1.15e-04	538.92	-1900.55
	(Gym) HalfCheetah	1.59e-04	59.74	6501.50
	(Gym) Hopper	1.46e-05	211.65	-783.52
	(Gym) Humanoid	2.44e-04	304.54	-351.25
	(Gym) Walker2d	3.32e-05	43.56	-1963.40

ates a learned policy network π :

$$Q(s, a) \approx r + \gamma Q(s', \pi(s')). \quad (6)$$

This value update evaluates the learned policy π rather than the action originally taken in the environment. When MPC is used to collect data, there is a mismatch in which the value function is trained on actions from π_{MPC} but queried on actions sampled from π . Because $\pi \neq \pi_{\text{MPC}}$, these queries are out-of-distribution, which prior work has shown leads to approximation errors and overestimated values (Fujimoto et al., 2019).

One might resolve this by using search to compute value targets directly, but doing so is prohibitively expensive. Every sample in a mini-batch requires a target, multiplying the number of costly search calls by the batch size and update-to-data ratio (e.g., $256 \times$ with default hyperparameters).

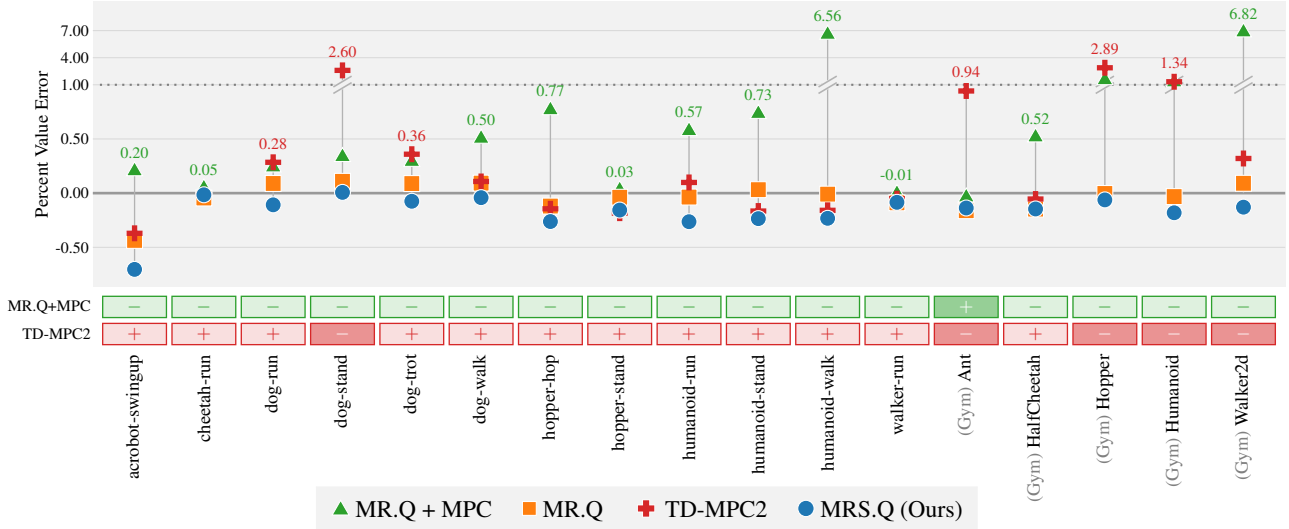


Figure 3. Value accuracy when using MPC. We report the percent error between value function estimates and true discounted returns at the end of training, where positive values indicate overestimation. Error bars represent standard deviation across 10 seeds. MR.Q+MPC replaces the behavior policy with MPC when acting in the environment. Reflecting Table 1, we mark environments where MPC improved (+) or harmed (-) MR.Q+MPC and TD-MPC2. Adding MPC to MR.Q significantly increases overestimation and harms performance, suggesting a correlation between overestimation and performance. In contrast, our approach (MRS.Q) uses MPC while significantly reducing overestimation bias.

To test this hypothesis, we measure the accuracy of value functions learned by MR.Q, MR.Q with MPC, TD-MPC2, and our proposed approach, MRS.Q (introduced in the following section) in Figure 3. Value error is computed by taking the difference between the learned value estimate and the true discounted return of the behavior policy: the learned policy network for MR.Q, and the MPC policy π_{MPC} for the others. Unlike prior work (Lin et al., 2025) which examined overestimation bias of the learned policy π , our experiments focus on the behavior policy (largely the MPC policy π_{MPC}).

Figure 3 shows that simply changing the behavior policy from the learned policy (MR.Q) to MPC (MR.Q+MPC) introduces significant overestimation bias. Moreover, Figure 3 reveals a clear relationship between value error and the performance change Δ from using MPC (Table 1):

- **MR.Q+MPC:** Switching from the learned policy to MPC introduces overestimation in nearly every environment. MPC improves performance only in Ant, where overestimation is the lowest.
- **TD-MPC2:** Compared to MR.Q+MPC, overestimation is considerably lower overall. However, it remains high in environments where TD-MPC2’s learned policy outperforms vanilla TD-MPC2 (dog-stand, Ant, Hopper, Humanoid, Walker2d).

These findings suggest that (1) MPC introduces significant overestimation bias, and (2) although the design choices in TD-MPC2 help mitigate this bias, they are insufficient as a standalone solution.

5. Effective Search in Model-based RL

Having identified overestimation bias as a key challenge limiting search effectiveness, we now present our approach. Our algorithm, **Model-based Representations for Search and Q-learning (MRS.Q)**, builds on MR.Q with targeted modifications that enable effective use of search.

MRS.Q introduces three key modifications to MR.Q: (1) search-based action selection with a short horizon, (2) aggressive minimization over an ensemble of value functions, and (3) minor hyperparameter changes.

Action selection. Following TD-MPC2, MRS.Q selects actions using Model Predictive Path Integral (MPPI) control (Williams et al., 2015) when acting in the environment. As such, we also use the same MPPI hyperparameters as TD-MPC2. Critically, this means that we adopt the same short MPC horizon of 3 steps, using the value function to estimate returns beyond this window. As discussed in Section 4.1, naive search can fail with high probability when planning over long horizons due to the expanded search space. A short horizon keeps the search space tractable and limits compounding error in the model rollout.

Minimum over value functions. Recent work addressing distribution shift in MBRL (Lin et al., 2025; Wang et al., 2025; Zhan et al., 2025) has focused on constraining the policy to match actions selected by search. However, this approach is problematic: search-selected actions shift rapidly during training as the model evolves.

In Figure 4, we compare the rate of change of actions se-

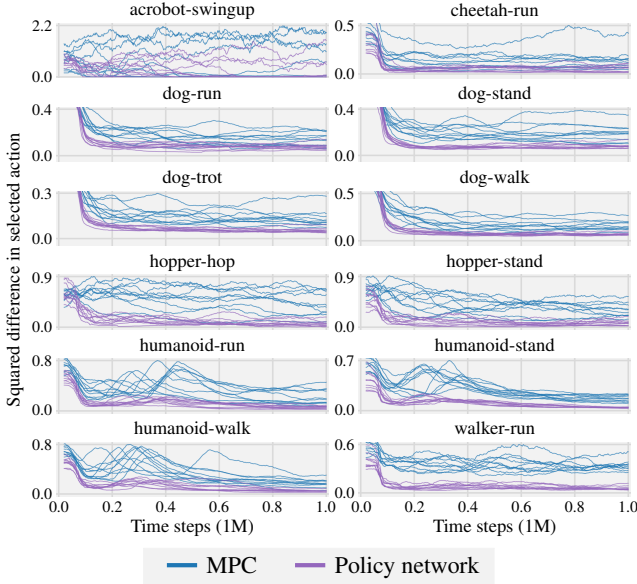


Figure 4. Change in selected action. We measure how actions selected by MPC and the policy network change over the course of learning. Specifically, we compute the mean-squared difference between selected actions at intervals of 1000 time steps, with all actions scaled to $[-1, 1]$. Across a wide range of environments, actions selected by MPC change significantly more than those selected by the policy network.

lected by search versus the learned policy network over the course of learning. Throughout training, search-selected actions exhibit substantially more variation than policy network actions, suggesting that constraining the policy to match such an unstable target may destabilize learning or introduce noise into training.

Rather than constraining or modifying the learned policy, we address distribution shift by directly reducing overestimation in the value function. We achieve this through aggressive minimization across an ensemble of 10 value functions. While taking the minimum over a pair of value functions has become a standard technique (Fujimoto et al., 2018), this approach naturally extends to larger ensembles:

$$Q(s, a) \approx r + \gamma \min_{i \in \{1, 2, \dots, 10\}} Q_i(s', \pi(s')). \quad (7)$$

Crucially, we apply this minimization wherever the value function is evaluated with the policy: not only when computing value targets (as above), but also when computing final values during MPC trajectory evaluation (Equation 3). Applying the minimum during trajectory evaluation prevents the search procedure from favoring potentially inflated value estimates over model-predicted rewards, while ensuring consistency in how the value function is used throughout the algorithm.

TD-MPC2 also employs multiple value functions ($N = 5$), however, rather than using the full ensemble, it ran-

domly samples a pair, taking the minimum for value updates (Chen et al., 2020) and the mean during MPC. As shown in Figure 3, using the minimum across the full ensemble significantly reduces overestimation compared to MR.Q ($N = 2$) naively combined with MPC. We further defend this choice in the ablation studies in Section 6.2.

Hyperparameter changes. In addition to the ensemble, we make only three hyperparameter changes to MR.Q:

- **No exploration noise.** As shown in Figure 4, search produces greater variation in action selection than the policy network. This variation in action selection implicitly induces exploration, making additive Gaussian noise unnecessary. We therefore remove it entirely (see Section 6.2).
- **Simplicial embedding (SEM).** Following TD-MPC2, we apply SEM (Lavoie et al., 2023; Obando-Ceron et al., 2026) to both the state encoder and the output of the dynamics model. SEM normalizes latent representations onto the probability simplex, which stabilizes multi-step dynamics rollouts by reducing drift in predicted states. Because SEM produces sparse representations with smaller loss magnitudes, we increase the dynamics loss weight from $1 \rightarrow 20$ to match the value used by TD-MPC2 and balance the loss terms.
- **Increased terminal loss weight.** Since predicting when episodes terminate is essential for accurate value estimation, we increase the weight of the terminal loss function from MR.Q’s default of $0.1 \rightarrow 1$.

All other hyperparameters and design choices follow the original MR.Q defaults. We highlight these minor adjustments to show that enabling effective search in MR.Q does not require major modifications to hyperparameter settings.

6. Results

We evaluate MRS.Q on a diverse set of continuous control tasks to demonstrate the performance benefits of MRS.Q.

Benchmarks. We evaluate on over 50 tasks spanning three benchmark domains: MuJoCo (Todorov et al., 2012; Towers et al., 2024), DeepMind Control Suite (DMC) (Tassa et al., 2018), and HumanoidBench (Sferrazza et al., 2024). For HumanoidBench, we split the results into two subsets, one which uses hand features, matching the experimental setting used by BMPC and BOOM (Wang et al., 2025; Zhan et al., 2025), and one which does not, matching the experimental setting used by SimbaV2 (Lee et al., 2025).

Baselines. We evaluate MRS.Q against several methods, including MR.Q (Fujimoto et al., 2025) (both with and without MPC), TD-MPC2 (Hansen et al., 2024), three state-of-the-art extensions of TD-MPC2 that regularize the policy to mitigate distribution shift: TD-M(PC)² (Lin et al.,

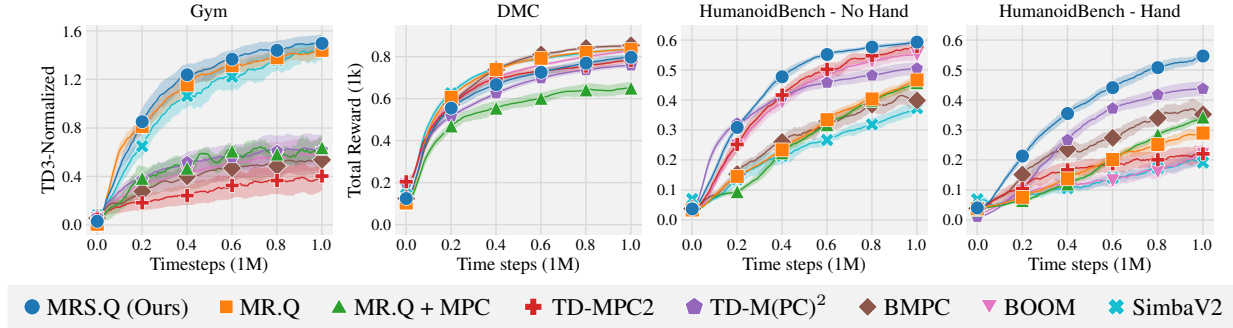


Figure 5. **Aggregate learning curves.** Our approach MRS.Q offers significant performance benefits over state-of-the-art model-based RL methods. The shaded area captures a 95% stratified bootstrap confidence interval across 10 seeds. Gym scores are normalized using TD3 performance values (Fujimoto et al., 2018) (see Appendix A.4 for details).

Table 2. **Aggregate Results.** Average final performance across each benchmark at 1M time steps. [Bracketed values] indicate 95% stratified bootstrap confidence intervals over 10 seeds. Gym scores are normalized to TD3 performance; other domains are scaled by dividing by 1k. The highest score per benchmark is highlighted.

Algorithm	MPC	Gym	DMC	HB (No Hand)	HB (Hand)
MR.Q	×	1.46 [1.41, 1.52]	0.84 [0.83, 0.84]	0.48 [0.46, 0.49]	0.31 [0.29, 0.32]
MR.Q + MPC	✓	0.67 [0.55, 0.88]	0.65 [0.63, 0.68]	0.46 [0.45, 0.48]	0.38 [0.37, 0.39]
TD-MPC2	✓	0.41 [0.27, 0.57]	0.78 [0.77, 0.80]	0.58 [0.56, 0.60]	0.22 [0.19, 0.25]
TD-M(PC) ²	✓	0.62 [0.50, 0.74]	0.76 [0.75, 0.77]	0.51 [0.48, 0.53]	0.44 [0.41, 0.46]
BMPC	✓	0.54 [0.44, 0.63]	<u>0.86 [0.86, 0.87]</u>	0.40 [0.37, 0.43]	0.38 [0.35, 0.40]
BOOM	✓	0.61 [0.47, 0.74]	0.83 [0.83, 0.84]	0.55 [0.53, 0.57]	0.23 [0.20, 0.26]
SimbaV2	×	1.44 [1.35, 1.52]	0.84 [0.83, 0.85]	0.38 [0.36, 0.40]	0.18 [0.17, 0.20]
MRS.Q	✓	<u>1.54 [1.46, 1.60]</u>	0.81 [0.79, 0.82]	<u>0.59 [0.58, 0.60]</u>	<u>0.58 [0.57, 0.58]</u>

2025), BMPC (Wang et al., 2025), and BOOM (Zhan et al., 2025), and the state-of-the-art model-free method SimbaV2 (Lee et al., 2025). All methods are trained for 1M environment steps (full details in Appendix A.5).

Evaluation protocol. For each task, we train agents for 1M environment steps and 10 seeds. We evaluate agents every 5k steps for 10 episodes without exploration noise. We report mean performance with 95% confidence intervals.

6.1. Main Results

We evaluate aggregate performance across all benchmarks, as shown in Figure 5 and Table 2. MRS.Q outperforms all baselines on three out of four benchmarks and remains competitive on the fourth (DMC). This demonstrates the effectiveness of our proposed modifications. While naively combining MPC with MR.Q degrades performance, our approach enables MRS.Q to fully leverage MPC.

Although TD-MPC2 was specifically designed for search and both methods use the same search procedure, MRS.Q achieves substantially stronger results, particularly on Gym and the higher-dimensional HumanoidBench tasks with hand features. This demonstrates that MRS.Q successfully unlocks MR.Q’s potential for search.

Furthermore, MRS.Q surpasses the TD-MPC2 extensions (TD-M(PC)², BMPC, BOOM) that regularize the policy towards search-selected actions. This suggests that simply imitating search behavior is insufficient to address distribution shift, likely due to the rapid evolution of selected actions during search (see Figure 4).

6.2. Ablation Studies

We ablate key components of MRS.Q to understand their individual contributions. Table 3 summarizes the results.

Number of value functions. We vary the number of value functions used in the ensemble. We find that performance improves as the ensemble size increases up to approximately 10 value functions, after which gains plateau. As shown in Figure 3, using the default 2 value functions is insufficient to address the overestimation induced by MPC, while an ensemble of 10 provides a good trade-off between bias reduction and computational cost.

Exploration noise. Re-adding exploration noise with MR.Q’s default of $\mathcal{N}(0, 0.2^2)$ induces a small performance drop across the four benchmarks. As illustrated in Figure 4, using MPC already induces substantial change in selected actions, making additional exploration noise on top of MPC unnecessary and often harmful.

Table 3. Ablation Studies. Average difference in normalized performance from varying design choices across each benchmark. [Bracketed values] indicate 95% stratified bootstrap confidence intervals over 5 seeds. Negative changes are highlighted with red, according to $[-0.01, -0.2]$, $[-0.2, -0.5]$, $(\leq -0.5]$. Positive changes are similarly highlighted with green (> 0.01) .

Ablation	Algorithm	Gym	DMC	HB (No Hand)	HB (Hand)
2 Value Functions	MRS.Q	-0.63 [-0.72, -0.53]	-0.04 [-0.05, -0.03]	-0.18 [-0.2, -0.16]	-0.40 [-0.41, -0.39]
5 Value Functions	MRS.Q	-0.37 [-0.47, -0.27]	0.00 [-0.01, 0.01]	-0.02 [-0.03, -0.01]	-0.13 [-0.14, -0.12]
20 Value Functions	MRS.Q	-0.05 [-0.10, -0.00]	-0.03 [-0.06, -0.01]	0.03 [0.01, 0.04]	0.03 [0.02, 0.04]
Exploration	MRS.Q	-0.13 [-0.24, -0.06]	-0.02 [-0.04, -0.00]	-0.02 [-0.03, -0.01]	-0.02 [-0.04, -0.01]
SEM	MRS.Q	-0.28 [-0.62, 0.05]	-0.04 [-0.06, -0.01]	0.06 [0.05, 0.07]	0.10 [0.10, 0.10]
Min (Ensemble)	MRS.Q	-0.49 [-0.55, -0.44]	0.01 [0.00, 0.03]	-0.05 [-0.07, -0.04]	-0.19 [-0.20, -0.18]
Min (MPC)	MRS.Q	-0.33 [-0.42, -0.19]	-0.02 [-0.04, -0.00]	-0.04 [-0.05, -0.03]	-0.19 [-0.21, -0.18]
Min (10)	TD-MPC2	0.07 [-0.08, 0.21]	0.03 [0.02, 0.05]	0.02 [0.00, 0.03]	0.11 [0.07, 0.15]

SEM. Replacing SEM in the state encoder with MR.Q’s default ELU activation function (Clevert et al., 2015) reduces performance on Gym and DMC, confirming that stabilizing latent dynamics is important for effective multi-step planning. However, this replacement slightly improves performance on HumanoidBench, suggesting that SEM is not the primary driver of performance gains in MRS.Q.

Minimum over ensemble. Instead of taking the minimum over the ensemble of value functions, we match TD-MPC2 by taking the minimum over two randomly sampled ensemble members (Chen et al., 2020). We find there is a significant performance loss in Gym and HumanoidBench - Hand. This suggests that overestimation bias mitigation is particularly important in high-dimensional tasks with early termination. Conversely, we remark this technique is not necessary in the DMC benchmark, possibly due to smoother dense rewards, and the lack of early termination.

Minimum in MPC. Our minimization strategy replaces all evaluations of the value function with a minimum over the entire ensemble. To measure the impact of minimization within MPC, we experiment with using the mean of the ensemble when evaluating trajectories during MPC, i.e., in Equation 3, as done in TD-MPC2. We find that this gives a moderate performance drop across the four benchmarks.

Minimum TD-MPC2. To evaluate the generalizability of our approach, we apply it to TD-MPC2 by taking the minimum over an ensemble of 10 value functions. We find that this improves performance across all four benchmarks.

7. Conclusion

In this work, we highlight a critical bottleneck in MBRL methods: the search process itself and the distribution shift it introduces. When value functions are trained using a non-search policy, but using search-gathered data, the resulting distribution shift leads to overestimation bias that undermines the benefits search is meant to provide. Our method, MRS.Q, adds search to MR.Q (Fujimoto et al., 2025) and

addresses this overestimation bias by taking the minimum over an ensemble of value functions. This targeted intervention gives consistent improvements over MR.Q and state-of-the-art model-based and model-free RL baselines on more than 50 tasks across popular benchmark domains.

More broadly, our results challenge a foundational assumption in MBRL: better models lead to better performance. Our analysis demonstrates that search can fail despite highly accurate, or even perfect, dynamics models, and that naively incorporating search into state-of-the-art algorithms can actively degrade performance. These findings show that model accuracy alone is insufficient and equal attention should be given to how models are used. By directly addressing the challenges of search, we can unlock performance gains that better models alone cannot provide.

Limitations. Despite MRS.Q’s strong performance, several limitations remain. First, the computational cost of maintaining 10 value functions and running MPC at each step is substantial. Although this cost is comparable to other MBRL methods such as TD-MPC2 (Hansen et al., 2024), it represents a significant increase over the search-free MR.Q (see Appendix B). Second, the aggressive minimization strategy provides smaller gains on tasks with dense rewards and no early termination (e.g., DMC), suggesting that the approach is most beneficial in settings where overestimation is severe. Lastly, because we adopt TD-MPC2’s search method, our approach inherits its short planning horizon, and our evaluation is limited to continuous control benchmarks. Nevertheless, our work offers novel insights into MBRL and highlights potential pitfalls of incorporating search into existing algorithms.

Impact Statement

Our contributions to sample-efficient MBRL have potential applications in robotics and autonomous systems, where data collection is costly, time-consuming, or safety-critical. By enabling more effective use of learned models, MRS.Q could reduce the real-world samples needed to train phys-

ical systems where extensive trial-and-error is impractical. While we evaluate only in simulation, the sample efficiency and performance gains demonstrated here may help accelerate the deployment of RL in real-world settings.

References

- Abbeel, P., Quigley, M., and Ng, A. Y. Using inaccurate models in reinforcement learning. In *Proceedings of the 23rd international conference on Machine learning*, pp. 1–8, 2006.
- Amos, B., Stanton, S., Yarats, D., and Wilson, A. G. On the model-based stochastic value gradient for continuous reinforcement learning. In *Learning for Dynamics and Control*, pp. 6–20. PMLR, 2021.
- An, G., Moon, S., Kim, J.-H., and Song, H. O. Uncertainty-based offline reinforcement learning with diversified q-ensemble. *Advances in neural information processing systems*, 34:7436–7447, 2021.
- Asadi, K., Misra, D., and Littman, M. Lipschitz continuity in model-based reinforcement learning. In *International conference on machine learning*, pp. 264–273. PMLR, 2018.
- Atkeson, C. G. and Santamaria, J. C. A comparison of direct and model-based reinforcement learning. In *Proceedings of international conference on robotics and automation*, volume 4, pp. 3557–3564. IEEE, 1997.
- Banijamali, E., Shu, R., Bui, H., Ghodsi, A., et al. Robust locally-linear controllable embedding. In *International Conference on Artificial Intelligence and Statistics*, pp. 1751–1759. PMLR, 2018.
- Brockman, G., Cheung, V., Pettersson, L., Schneider, J., Schulman, J., Tang, J., and Zaremba, W. Openai gym, 2016.
- Buckman, J., Hafner, D., Tucker, G., Brevdo, E., and Lee, H. Sample-efficient reinforcement learning with stochastic ensemble value expansion. In *Advances in Neural Information Processing Systems*, pp. 8234–8244, 2018.
- Chen, X., Wang, C., Zhou, Z., and Ross, K. W. Randomized ensembled double q-learning: Learning fast without a model. In *International Conference on Learning Representations*, 2020.
- Chua, K., Calandra, R., McAllister, R., and Levine, S. Deep reinforcement learning in a handful of trials using probabilistic dynamics models. In *Advances in Neural Information Processing Systems 31*, pp. 4759–4770, 2018.
- Clevert, D.-A., Unterthiner, T., and Hochreiter, S. Fast and accurate deep network learning by exponential linear units (elus). *arXiv preprint arXiv:1511.07289*, 2015.
- Deisenroth, M. and Rasmussen, C. E. Pilco: A model-based and data-efficient approach to policy search. In *International Conference on Machine Learning*, pp. 465–472, 2011.
- Draeger, A., Engell, S., and Ranke, H. Model predictive control using neural networks. *IEEE Control Systems Magazine*, 15(5):61–66, 1995.
- Ebert, F., Finn, C., Lee, A. X., and Levine, S. Self-supervised visual planning with temporal skip connections. *Conference on Robot Learning*, 2017.
- Farebrother, J., Pirotta, M., Tirinzoni, A., Munos, R., Lazaric, A., and Touati, A. Temporal difference flows. In *Forty-second International Conference on Machine Learning*, 2025.
- Feinberg, V., Wan, A., Stoica, I., Jordan, M. I., Gonzalez, J. E., and Levine, S. Model-based value estimation for efficient model-free reinforcement learning. *arXiv preprint arXiv:1803.00101*, 2018.
- Finn, C., Tan, X. Y., Duan, Y., Darrell, T., Levine, S., and Abbeel, P. Deep spatial autoencoders for visuomotor learning. In *2016 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 512–519. IEEE, 2016.
- Fujimoto, S., van Hoof, H., and Meger, D. Addressing function approximation error in actor-critic methods. In *International Conference on Machine Learning*, volume 80, pp. 1587–1596. PMLR, 2018.
- Fujimoto, S., Meger, D., and Precup, D. Off-policy deep reinforcement learning without exploration. In *International Conference on Machine Learning*, pp. 2052–2062, 2019.
- Fujimoto, S., Meger, D., and Precup, D. An equivalence between loss functions and non-uniform sampling in experience replay. *Advances in Neural Information Processing Systems*, 33, 2020.
- Fujimoto, S., Chang, W.-D., Smith, E. J., Gu, S. S., Precup, D., and Meger, D. For SALE: State-action representation learning for deep reinforcement learning. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2024.
- Fujimoto, S., D’Oro, P., Zhang, A., Tian, Y., and Rabbat, M. Towards general-purpose model-free reinforcement learning. In *The Thirteenth International Conference on Learning Representations*, 2025.

- Gal, Y., McAllister, R., and Rasmussen, C. E. Improving pilco with bayesian neural network dynamics models. In *Data-Efficient Machine Learning workshop, International Conference on Machine Learning*, 2016.
- Glorot, X. and Bengio, Y. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the thirteenth international conference on artificial intelligence and statistics*, pp. 249–256. JMLR Workshop and Conference Proceedings, 2010.
- Gu, S., Lillicrap, T., Sutskever, I., and Levine, S. Continuous deep q-learning with model-based acceleration. In *International conference on machine learning*, pp. 2829–2838. PMLR, 2016.
- Ha, D. and Schmidhuber, J. Recurrent world models facilitate policy evolution. *Advances in neural information processing systems*, 31, 2018.
- Hafner, D., Lillicrap, T., Fischer, I., Villegas, R., Ha, D., Lee, H., and Davidson, J. Learning latent dynamics for planning from pixels. In *International conference on machine learning*, pp. 2555–2565. PMLR, 2019.
- Hafner, D., Pasukonis, J., Ba, J., and Lillicrap, T. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.
- Hansen, N., Su, H., and Wang, X. Td-mpc2: Scalable, robust world models for continuous control. In *The Twelfth International Conference on Learning Representations*, 2024.
- Hansen, N. A., Su, H., and Wang, X. Temporal difference learning for model predictive control. In *International Conference on Machine Learning*, pp. 8387–8406. PMLR, 2022.
- Henaff, M., Canziani, A., and LeCun, Y. Model-predictive policy learning with uncertainty regularization for driving in dense traffic. In *International Conference on Learning Representations*, 2019.
- Higuera, J. C. G., Meger, D., and Dudek, G. Synthesizing neural network controllers with probabilistic model-based reinforcement learning. In *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 2538–2544, 2018.
- Janner, M., Fu, J., Zhang, M., and Levine, S. When to trust your model: Model-based policy optimization. *Advances in Neural Information Processing Systems*, 32, 2019.
- Janner, M., Du, Y., Tenenbaum, J., and Levine, S. Planning with diffusion for flexible behavior synthesis. In *International Conference on Machine Learning*, pp. 9902–9915. PMLR, 2022.
- Lambert, N., Amos, B., Yadan, O., and Calandra, R. Objective mismatch in model-based reinforcement learning. In *Learning for Dynamics and Control*, pp. 761–770. PMLR, 2020.
- Lambert, N., Wilcox, A., Zhang, H., Pister, K. S., and Calandra, R. Learning accurate long-term dynamics for model-based reinforcement learning. In *2021 60th IEEE Conference on decision and control (CDC)*, pp. 2880–2887. IEEE, 2021.
- Lambert, N., Pister, K., and Calandra, R. Investigating compounding prediction errors in learned dynamics models. *arXiv preprint arXiv:2203.09637*, 2022.
- Lan, Q., Pan, Y., Fyshe, A., and White, M. Maxmin q-learning: Controlling the estimation bias of q-learning. In *International Conference on Learning Representations*, 2020.
- Lavoie, S., Tsirigotis, C., Schwarzer, M., Vani, A., Noukhovitch, M., Kawaguchi, K., and Courville, A. Simplicial embeddings in self-supervised learning and downstream classification. In *The Eleventh International Conference on Learning Representations*, 2023.
- Lee, H., Lee, Y., Seno, T., Kim, D., Stone, P., and Choo, J. Hyperspherical normalization for scalable deep reinforcement learning. In *Forty-second International Conference on Machine Learning*, 2025.
- Levine, S. and Koltun, V. Guided policy search. In *International conference on machine learning*, pp. 1–9. PMLR, 2013.
- Lillicrap, T. P., Hunt, J. J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D., and Wierstra, D. Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971*, 2015.
- Lin, H., Wang, P., Schneider, J., and Shi, G. Td-m(pc)²: Improving temporal difference mpc through policy constraint. *arXiv preprint arXiv:2502.03550*, 2025.
- Loshchilov, I. and Hutter, F. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.
- Lowrey, K., Rajeswaran, A., Kakade, S., Todorov, E., and Mordatch, I. Plan online, learn offline: Efficient learning and exploration via model-based control. In *International Conference on Learning Representations*, 2019.
- Ma, M., Ni, T., Gehring, C., D’Oro, P., and Bacon, P.-L. Do transformer world models give better policy gradients? In *International Conference on Machine Learning*, pp. 33855–33879. PMLR, 2024.

- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., et al. Human-level control through deep reinforcement learning. *Nature*, 518 (7540):529–533, 2015.
- Mordatch, I. and Todorov, E. Combining the benefits of function approximation and trajectory optimization. In *Robotics: Science and Systems*, volume 4, pp. 23, 2014.
- Nagabandi, A., Kahn, G., Fearing, R. S., and Levine, S. Neural network dynamics for model-based deep reinforcement learning with model-free fine-tuning. In *2018 IEEE international conference on robotics and automation (ICRA)*, pp. 7559–7566. IEEE, 2018.
- Obando-Ceron, J., Mayor, W., Lavoie, S., Fujimoto, S., Courville, A., and Castro, P. S. Simplicial embeddings improve sample efficiency in actor-critic agents. In *The Fourteenth International Conference on Learning Representations*, 2026.
- Oh, J., Guo, X., Lee, H., Lewis, R. L., and Singh, S. Action-conditional video prediction using deep networks in atari games. *Advances in neural information processing systems*, 28, 2015.
- Oh, J., Singh, S., and Lee, H. Value prediction network. *Advances in neural information processing systems*, 30, 2017.
- Palenicek, D., Lutter, M., Carvalho, J., and Peters, J. Diminishing return of value expansion methods in model-based reinforcement learning. In *The Eleventh International Conference on Learning Representations*, 2023.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., et al. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, pp. 8024–8035, 2019.
- Schrittwieser, J., Antonoglou, I., Hubert, T., Simonyan, K., Sifre, L., Schmitt, S., Guez, A., Lockhart, E., Hassabis, D., Graepel, T., et al. Mastering atari, go, chess and shogi by planning with a learned model. *Nature*, 588 (7839):604–609, 2020.
- Schrittwieser, J., Hubert, T., Mandhane, A., Barekatin, M., Antonoglou, I., and Silver, D. Online and offline reinforcement learning by planning with a learned model. *Advances in Neural Information Processing Systems*, 34: 27580–27591, 2021.
- Schwarzer, M., Ceron, J. S. O., Courville, A., Bellemare, M. G., Agarwal, R., and Castro, P. S. Bigger, better, faster: Human-level atari with human-level efficiency. In *International Conference on Machine Learning*, pp. 30365–30380. PMLR, 2023.
- Sferrazza, C., Huang, D.-M., Lin, X., Lee, Y., and Abbeel, P. HumanoidBench: Simulated Humanoid Benchmark for Whole-Body Locomotion and Manipulation. In *Proceedings of Robotics: Science and Systems*, 2024.
- Silver, D., Hasselt, H., Hessel, M., Schaul, T., Guez, A., Harley, T., Dulac-Arnold, G., Reichert, D., Rabinowitz, N., Barreto, A., et al. The predictron: End-to-end learning and planning. In *International Conference on Machine Learning*, pp. 3191–3199. PMLR, 2017.
- Sutton, R. S. Dyna, an integrated architecture for learning, planning, and reacting. *ACM Sigart Bulletin*, 2(4):160–163, 1991.
- Sutton, R. S. and Barto, A. G. *Reinforcement Learning: An Introduction*, volume 1. MIT press Cambridge, 1998.
- Talvitie, E. Model regularization for stable sample rollouts. In *UAI*, pp. 780–789, 2014.
- Tassa, Y., Doron, Y., Muldal, A., Erez, T., Li, Y., Casas, D. d. L., Budden, D., Abdolmaleki, A., Merel, J., Lefrancq, A., et al. Deepmind control suite. *arXiv preprint arXiv:1801.00690*, 2018.
- Thrun, S. and Schwartz, A. Issues in using function approximation for reinforcement learning. In *Proceedings of the 1993 Connectionist Models Summer School Hillsdale, NJ. Lawrence Erlbaum*, 1993.
- Todorov, E., Erez, T., and Tassa, Y. Mujoco: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 5026–5033. IEEE, 2012.
- Towers, M., Kwiatkowski, A., Terry, J., Balis, J. U., De Cola, G., Deleu, T., Goulão, M., Kallinteris, A., Krimmel, M., KG, A., et al. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024.
- Van Hasselt, H. Double q-learning. In *Advances in Neural Information Processing Systems*, pp. 2613–2621, 2010.
- Van Hasselt, H., Guez, A., and Silver, D. Deep reinforcement learning with double q-learning. In *AAAI*, pp. 2094–2100, 2016.
- Van Hasselt, H. P., Hessel, M., and Aslanides, J. When to use parametric models in reinforcement learning? *Advances in Neural Information Processing Systems*, 32, 2019.
- Venkatraman, A., Hebert, M., and Bagnell, J. Improving multi-step prediction of learned time series models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 29, 2015.

- Wang, S., Liu, S., Ye, W., You, J., and Gao, Y. Efficientzero v2: Mastering discrete and continuous control with limited data. In *International Conference on Machine Learning*, pp. 51041–51062. PMLR, 2024.
- Wang, Y., Guo, H., Wang, S., Qian, L., and Lan, X. Bootstrapped model predictive control. In *The Thirteenth International Conference on Learning Representations*, 2025.
- Watter, M., Springenberg, J., Boedecker, J., and Riedmiller, M. Embed to control: A locally linear latent dynamics model for control from raw images. *Advances in neural information processing systems*, 28, 2015.
- Williams, G., Aldrich, A., and Theodorou, E. Model predictive path integral control using covariance variable importance sampling. *arXiv preprint arXiv:1509.01149*, 2015.
- Ye, W., Liu, S., Kurutach, T., Abbeel, P., and Gao, Y. Mastering atari games with limited data. *Advances in neural information processing systems*, 34:25476–25488, 2021.
- Zhan, G., Wang, L., Zhang, X., Gao, J., Tomizuka, M., and Li, S. E. Bootstrap off-policy with world model. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.

Appendix

A	Experimental Details	14
A.1	Hyperparameters	14
A.2	MPC Procedure	16
A.3	Network Architecture	18
A.4	Environments	20
A.5	Baselines	20
A.6	Model and Value Accuracy Experiments	21
B	Computational Cost	22
C	Full Results	23
C.1	Gym	23
C.2	DMC	24
C.3	HumanoidBench (No Hand)	26
C.4	HumanoidBench (Hand)	28
D	Ablation Results	30

A. Experimental Details

A.1. Hyperparameters

Table 4. **MRS.Q Hyperparameters.** New or changed hyperparameters from MR.Q are highlighted. Hyperparameters values are kept fixed across all benchmarks. MPC hyperparameters come directly from TD-MPC2 (Hansen et al., 2024), except for the policy standard deviation.

	Hyperparameter	Value
MR.Q (Fujimoto et al., 2025)	Dynamics loss weight $\lambda_{\text{Dynamics}}$	$1 \rightarrow 20$
	Reward loss weight λ_{Reward}	0.1
	Terminal loss weight $\lambda_{\text{Terminal}}$	$0.1 \rightarrow 1$
	Pre-activation loss weight $\lambda_{\text{pre-activ}}$	$1e - 5$
	Encoder horizon H_{Enc}	5
	Multi-step returns horizon H_Q	3
TD3 (Fujimoto et al., 2018)	Target policy noise σ	$\mathcal{N}(0, 0.2^2)$
	Target policy noise clipping c	$(-0.3, 0.3)$
LAP (Fujimoto et al., 2020)	Probability smoothing α	0.4
	Minimum priority	1
Exploration	Initial random exploration time steps	10k
	Exploration noise	$\mathcal{N}(0, 0.2^2) \rightarrow \mathcal{N}(0, 0)$
Common	Discount factor γ	0.99
	Replay buffer capacity	1M
	Mini-batch size	256
	Target update frequency T_{target}	250
	Replay ratio	1
Encoder Network	Optimizer	AdamW (Loshchilov & Hutter, 2019)
	Learning rate	$1e - 4$
	Weight decay	$1e - 4$
	$\mathbf{z}_s$ dim	512
	$\mathbf{z}_{sa}$ dim	512
	$\mathbf{z}_a$ dim (only used within architecture)	256
	Hidden dim	512
	Activation function	ELU (Clevert et al., 2015)
	Weight initialization	Xavier uniform (Glorot & Bengio, 2010)
	Bias initialization	0
	Reward bins	65
	Reward range	$[-10, 10]$ (effective: $[-22k, 22k]$)
	SEM Groups	8
Value Network	Optimizer	AdamW
	Learning rate	$3e - 4$
	Hidden dim	512
	Activation function	ELU
	Weight initialization	Xavier uniform
	Bias initialization	0
	Gradient clip norm	20
	Ensemble size	$2 \rightarrow 10$
Policy Network	Optimizer	AdamW
	Learning rate	$3e - 4$
	Hidden dim	512
	Activation function	ReLU
	Weight initialization	Xavier uniform
	Bias initialization	0
MPC (Williams et al., 2015) (Hansen et al., 2024)	MPC horizon H	3
	Number of MPC iterations I	6
	Number of samples n	512
	Number of policy actions n_π	24
	Number of elites k	64
	Policy standard deviation σ_{det}	0.1
	Max standard deviation σ_{max}	2
	Min standard deviation σ_{min}	0.05
	Temperature τ	0.5

A.2. MPC Procedure

For our search algorithm, we follow the TD-MPC2 (Hansen et al., 2024) version of Model Predictive Path Integral (MPPI) control (Williams et al., 2015). Let $f(s) = \mathbf{z}_s$ be the state encoder, let $g(\mathbf{z}_s, a) = \mathbf{z}_{sa}$ be the state-action encoder, and let $h(\mathbf{z}_{sa})$ be the dynamics prediction model. Let t_{episode} denote the current time step of the episode.

Algorithm 1 MPC

Input

- State s

Hyperparameters

- MPC horizon H
 - Number of MPC iterations I
 - Number of samples n
 - Number of policy actions n_π
 - Number of elites k
 - Policy standard deviation σ_{det}
 - Max standard deviation σ_{max}
 - Min standard deviation σ_{min}
 - Temperature τ
-

Sample policy actions

$\mathbf{z}_s = f(s)$

Encode initial state

For $t = 0$ **to** $H - 1$ **do**:

$a_t^\pi = \{\mathcal{N}(\pi(\mathbf{z}_s), \sigma_{\text{det}})\}_{n_\pi}$

a_t^π is a set of n_π actions

$\mathbf{z}_s = h(g(\mathbf{z}_s, a_t^\pi))$

Roll the model forward

Initialize mean and standard deviation of randomly sampled actions

$\mu_t = \mu_{t+1}^{\text{prev}}$ **if** $t_{\text{episode}} > 0$ **else** $\mu_t = 0$

Initialize random action mean using results from a prior MPPI call

$\sigma_t = \sigma_{\text{max}}$

Initialize standard deviation

MPC iterations

For $i = 0$ **to** $I - 1$ **do**:

$a_t^{\text{rand}} = \{N(\mu_t, \sigma_t)\}_{n-n_\pi}$

a_t^{rand} is a set of $n - n_\pi$ actions

$a_t = [a_t^\pi, a_t^{\text{rand}}]$

Actions are both the policy actions and the sampled actions

$V = \text{EstimateValue}(s, a_t)$

Compute the value of each action sequence $\{a_0, a_1, \dots\}$

$V_{\text{elite}}, a_{\text{elite}} = \text{Top-k}(V, k)$

Select the highest k valued actions

$s = e^{\tau(V_{\text{elite}} - \max V_{\text{elite}})}$

Score of each action, using their value and temperature τ

$s \leftarrow \sum s$

Normalize scores

$\mu = \sum s a_{\text{elite}}$

Compute new mean

$\sigma = \sqrt{\sum s(a_{\text{elite}} - \mu)^2}$

Compute new standard deviation

$\sigma = \text{clamp}(\sigma, \sigma_{\text{min}}, \sigma_{\text{max}})$

$\mu_{\text{prev}} = \mu$

Save mean for MPC call at the next episode time step

Return $\text{GumbelSoftMaxSample}(a_{\text{elite}}, s, \tau)$

Sample final action based on scores

Algorithm 2 EstimateValue

Input

- State s
- Sequence of actions a_t

Hyperparameters

- MPC horizon H
 - Discount γ
-

$R = 0, d = 1, g = 1$	Initialize tracking variables
$\mathbf{z}_s = f(s)$	Encode initial state
For $t = 0$ to $H - 1$ do :	
$\mathbf{z}_s, \tilde{r}, \tilde{d} = h(g(\mathbf{z}_s, a_t))$	Roll the model forward, predicting next embedding $\mathbf{z}_s$, reward $\tilde{r}$, and termination $\tilde{d}$
$R = R + g\tilde{r}$	Update accumulated return
$d = 1 - \text{round}(\tilde{d})$	Update accumulated termination, rounding to make predicted termination $\in \{0, 1\}$
$g = \gamma g$	Update accumulated discount
$a_H = \pi(\mathbf{z}_s)$	Final policy action
$Q = \min_i Q_i(g(\mathbf{z}_s, a_H))$	Final value, taking minimum over the ensemble
Return $R + gdQ$	Compute and return the final value of the action sequence

A.3. Network Architecture

This section describes MRS.Q’s network architecture using PyTorch code blocks (Paszke et al., 2019). The network definitions follow MR.Q’s architecture, with minor modifications to incorporate SEM (Lavoie et al., 2023).

Preamble

```

1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4
5 zs_dim = 512
6 za_dim = 256
7 zsa_dim = 512
8
9 def ln(x):
10     return F.layer_norm(x, (x.shape[-1],))
11
12 def SEM(self, x):
13     shape = x.shape
14     x = x.reshape(*shape[:-1], -1, 8)
15     x = F.softmax(x, dim=-1)
16     return x.reshape(*shape)
    
```

State Encoder f Network

The state encoder is a three-layer MLP with a hidden dimension of 512. Each hidden layer is followed by LayerNorm and an ELU activation. The final layer applies LayerNorm with a learnable affine transformation, followed by SEM.

The resulting state embedding $\mathbf{z}_s$ is trained end-to-end with the state-action encoder.

```

1 self.zs1 = nn.Linear(state_dim, 512)
2 self.zs2 = nn.Linear(512, 512)
3 self.zs3 = nn.Linear(512, zs_dim)
4 self.ln = nn.LayerNorm(zs_dim)
5
6 self.activ = F.elu
7
8 def forward(self, state):
9     zs = self.activ(ln(self.zs1(state)))
10    zs = self.activ(ln(self.zs2(zs)))
11    return SEM(self.ln(self.zs3(zs)))
    
```

State-Action Encoder g Network

The action input passes through a linear layer followed by an ELU activation. The processed action is then concatenated with the state embedding and fed through a three-layer MLP with a hidden dimension of 512. LayerNorm and ELU activations follow the first two layers.

The resulting state-action embedding $\mathbf{z}_{sa}$ is passed to a linear layer that predicts the next state embedding, reward, and terminal signal. The final layer of the dynamics model applies LayerNorm with a learnable affine transformation, followed by SEM.

```

1 self.za = nn.Linear(action_dim, za_dim)
2 self.zsa1 = nn.Linear(zs_dim + za_dim, 512)
3 self.zsa2 = nn.Linear(512, 512)
4 self.zsa3 = nn.Linear(512, zsa_dim)
5
6 self.ln = nn.LayerNorm(zs_dim)
7 self.dynamics_model = nn.Linear(zsa_dim, zs_dim)
8 self.reward_model = nn.Linear(zsa_dim, 65)
9 self.terminal_model = nn.Linear(zsa_dim, 1)
10
11 self.activ = F.elu
12
    
```

```

13 def forward(self, zs, action):
14     za = self.activ(self.za(action))
15     zsa = torch.cat([zs, za], 1)
16     zsa = self.activ(ln(self.zsa1(zsa)))
17     zsa = self.activ(ln(self.zsa2(zsa)))
18     zsa = self.zsa3(zsa)
19     return (
20         SEM(self.ln(self.dynamics_model(zsa))),
21         self.reward_model(zsa),
22         self.terminal_model(zsa),
23         zsa
24     )

```

Value Q Networks

The value network is a four-layer MLP with a hidden dimension of 512. LayerNorm and ELU activations follow the first three layers. An ensemble of 10 value networks is used, each sharing the same architecture and forward pass.

```

1 self.l1 = nn.Linear(zsa_dim, 512)
2 self.l2 = nn.Linear(512, 512)
3 self.l3 = nn.Linear(512, 512)
4 self.l4 = nn.Linear(512, 1)
5
6 self.activ = F.elu
7
8 def forward(self, zsa):
9     q = self.activ(ln(self.l1(zsa)))
10    q = self.activ(ln(self.l2(q)))
11    q = self.activ(ln(self.l3(q)))
12    return self.l4(q)

```

Policy π Network

The policy network is a three-layer MLP with a hidden dimension of 512. LayerNorm and ReLU activations follow the first two layers, with a tanh function applied to the output.

```

1 self.l1 = nn.Linear(zs_dim, 512)
2 self.l2 = nn.Linear(hdim, 512)
3 self.l3 = nn.Linear(512, action_dim)
4
5 self.activ = F.relu
6
7 def forward(self, zs):
8     a = self.activ(ln(self.l1(zs)))
9     a = self.activ(ln(self.l2(a)))
10    return torch.tanh(self.l3(a))

```

A.4. Environments

All main experiments for MRS.Q and baseline methods used 10 random seeds, while ablation studies used 5 seeds. Evaluations were performed every 5k time steps, taking the average performance across 10 episodes.

Gym (Todorov et al., 2012; Brockman et al., 2016; Towers et al., 2024). We evaluate on the same five environments used by MR.Q (Fujimoto et al., 2025), all based on the $-v4$ version with no preprocessing applied. Following MR.Q, we report aggregate scores using TD3-normalized performance, with TD3 reference scores obtained from TD7 (Fujimoto et al., 2024):

$$\text{TD3-Normalized}(x) := \frac{x - \text{random score}}{\text{TD3 score} - \text{random score}}. \quad (8)$$

Table 5. Scores used to normalize Gym tasks.

	Random	TD3
Ant	-70.288	3942
HalfCheetah	-289.415	10574
Hopper	18.791	3226
Humanoid	120.423	5165
Walker2d	2.791	3946

DM Control Suite (DMC) (Tassa et al., 2018). We evaluate on the same 28 environments used by MR.Q, with an action repeat of 2 following prior work (Hansen et al., 2024; Fujimoto et al., 2025).

HumanoidBench (Sferrazza et al., 2024). We evaluate on the same set of 14 $-v0$ environments for both HumanoidBench experiment configurations. Following prior work (Wang et al., 2025), we omit the reach task from aggregate scores due to its different reward scale. Experiments without hands use the `h1` environments (e.g., `h1-walk-v0`), while experiments with hands use the `h1-hand` environments (e.g., `h1hand-walk-v0`). For action repeat settings, MRS.Q and the model-based baselines (TD-MPC2, BMPC, BOOM) use an action repeat of 1, following author-provided hyperparameters. The model-free baselines (MR.Q, SimbaV2) use an action repeat of 2; MR.Q+MPC also uses an action repeat of 2 to ensure a fair comparison with MR.Q.

A.5. Baselines

All experiments were run for 10 seeds over 1M environment steps and use default author-provided hyperparameters for all tasks.

MR.Q (Fujimoto et al., 2025). Results were obtained from the authors’ GitHub repository (<https://github.com/facebookresearch/MRQ>), commit 280d9c0263964463522d16ec2daee57b1b7bf087, except for HumanoidBench, where we re-ran the authors’ code. For HumanoidBench, we use an action repeat of 2, as this achieved a slightly higher performance than an action repeat of 1.

TD-MPC2 (Hansen et al., 2024). Results were obtained by re-running the authors’ code (<https://github.com/nicklashansen/tdmpc2>, commit 8bbc14ebabdb32ea7ada5c801dc525d0dc73baf6). Unlike the MR.Q paper, which used separate TD-MPC2 codebases for terminal and non-terminal environments, we use this single codebase for all experiments, as it is designed to handle both settings.

TD-M(PC)² (Lin et al., 2025). Results were obtained by re-running the authors’ code (https://github.com/DarthUtopian/tdmpc_square_public), commit d1c2632c36effd2f7b661bfe5f822a3db8054d40. To enable termination prediction for Gym tasks, we extend the dynamics model to include termination signals, following the TD-MPC2 implementation. This modification applies only to Gym tasks with termination conditions.

BMPC (Wang et al., 2025). Results were obtained by re-running the authors’ code (<https://github.com/wertyuilife2/bmpc>), commit 6746ea7d265a8b82c8347fd7e894373ce00333fa. To enable termination prediction for Gym tasks, we extend the dynamics model to include termination signals, following the TD-MPC2 implementation. This modification applies only to Gym tasks with termination conditions.

BOOM (Zhan et al., 2025). Results were obtained by re-running the authors’ code (https://github.com/molunitu/BOOM_MBRL), commit 9b3156d1fadda5ca5318274c404b694ad7b0786f. To enable termination prediction

for Gym tasks, we extend the dynamics model to include termination signals, following the TD-MPC2 implementation. This modification applies only to Gym tasks with termination conditions.

SimbaV2 (Lee et al., 2025). Results were obtained by re-running the authors’ code (<https://github.com/DAVIAN-Robotics/SimbaV2>), commit 86899c277cdc697b2b02d827243de1ea93f20a1d.

A.6. Model and Value Accuracy Experiments

To compute the error terms reported in Table 1 and Figure 3, we evaluate error terms on state-action pairs sampled according to a fully trained agent. Specifically, for each algorithm, we use the final policy to collect 50 state-action pairs along a trajectory at regular intervals of 20 time steps, resetting the environment as needed.

We obtain ground-truth value estimates by resetting the internal simulator to each recorded state, executing the initial action, and then following the agent’s policy until episode termination. We repeat this process 100 times per state-action pair to ensure reliable estimates.

B. Computational Cost

We evaluate the computational cost of MRS.Q against our baselines using consistent hardware, with results reported in [Table 6](#). For a fair comparison, `torch.compile` is disabled for all approaches. All experiments are conducted on HalfCheetah-v4 for 1M time steps.

Table 6. Computational cost.

Method	MPC	Wall-clock time
MR.Q	×	10 hours
MR.Q + 10 value functions	×	16 hours
TD-MPC2	✓	45 hours
MRS.Q	✓	53 hours

The results show that although MRS.Q incurs significantly higher computational cost than MR.Q, this overhead is primarily due to MPC rather than the ensemble of 10 value functions.

C. Full Results

C.1. Gym

Table 7. Gym results. Average final performance at 1M time steps. [Bracketed values] indicate 95% stratified bootstrap confidence intervals over 10 seeds.

Task	MRS.Q	MR.Q	MR.Q+MPC	TD-MPC2	TD-M(PC) ²	BMPC	BOOM	SimbaV2
Ant	7189 [6394, 7804]	6901 [6261, 7482]	7068 [6627, 7865]	1938 [795, 3694]	3516 [1899, 5336]	5102 [2664, 6966]	4740 [3206, 6167]	6912 [6545, 7192]
HalfCheetah	14649 [14149, 15158]	12939 [11663, 13762]	4544 [1083, 9189]	11188 [6871, 15475]	12419 [10716, 13776]	14319 [11251, 16002]	12289 [8333, 16247]	13168 [13039, 13281]
Hopper	3248 [2945, 3516]	2692 [2131, 3309]	702 [141, 1313]	267 [170, 360]	334 [232, 439]	32 [8, 64]	196 [143, 253]	3483 [2972, 3948]
Humanoid	10526 [10186, 10816]	10223 [9929, 10498]	4531 [868, 9729]	297 [260, 336]	3003 [1724, 4239]	220 [182, 258]	1949 [1046, 3480]	9326 [8213, 10339]
Walker2d	5699 [5304, 5981]	6039 [5644, 6386]	169 [33, 439]	1537 [668, 2592]	1452 [564, 2561]	145 [51, 244]	1005 [688, 1506]	5214 [3821, 6323]
Mean	1.54 [1.46, 1.60]	1.46 [1.41, 1.52]	0.67 [0.55, 0.88]	0.41 [0.27, 0.57]	0.62 [0.50, 0.74]	0.54 [0.44, 0.63]	0.61 [0.47, 0.74]	1.44 [1.35, 1.52]
Median	1.44 [1.36, 1.52]	1.53 [1.43, 1.61]	0.44 [0.15, 0.62]	0.39 [0.17, 0.65]	0.57 [0.35, 0.79]	0.04 [0.01, 0.06]	0.36 [0.23, 0.67]	1.32 [1.23, 1.60]
IQM	1.54 [1.45, 1.62]	1.50 [1.44, 1.55]	0.51 [0.35, 0.81]	0.32 [0.15, 0.53]	0.61 [0.43, 0.79]	0.45 [0.26, 0.51]	0.59 [0.42, 0.70]	1.43 [1.33, 1.54]

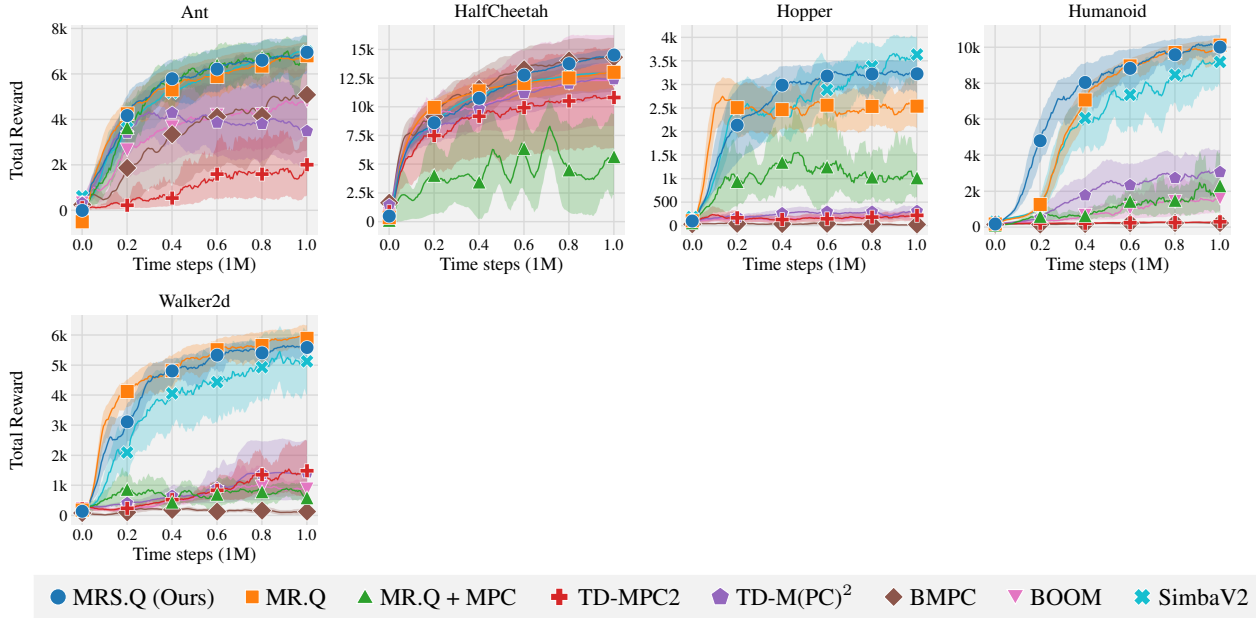


Figure 8. Gym learning curves. The shaded area captures a 95% stratified bootstrap confidence interval across 10 seeds.

C.2. DMC

 Table 8. **DMC results.** Average final performance at 1M time steps. [Bracketed values] indicate 95% stratified bootstrap confidence intervals over 10 seeds.

Task	MRS.Q	MR.Q	MR.Q+MPC	TD-MPC2	TD-M(PC) ²	BMPC	BOOM	SimbaV2
acrobot-swingup	468 [417, 515]	567 [517, 621]	459 [264, 598]	554 [391, 690]	592 [550, 630]	550 [503, 599]	619 [600, 638]	497 [424, 578]
ball.in.cup-catch	979 [977, 982]	981 [979, 983]	976 [972, 982]	979 [973, 986]	984 [982, 986]	983 [981, 986]	984 [982, 986]	982 [981, 985]
cartpole-balance	995 [992, 998]	999 [999, 1000]	974 [924, 1000]	998 [998, 999]	996 [993, 998]	993 [987, 998]	996 [995, 999]	999 [1000, 1000]
cartpole-balance-sparse	1000 [1000, 1000]	1000 [1000, 1000]	1000 [1000, 1000]	970 [912, 1000]	999 [998, 1000]	997 [993, 1000]	1000 [1000, 1000]	935 [807, 1000]
cartpole-swingup	866 [865, 867]	866 [866, 866]	687 [527, 830]	879 [875, 882]	877 [874, 881]	881 [881, 882]	878 [874, 882]	881 [882, 882]
cartpole-swingup-sparse	623 [424, 788]	798 [779, 818]	608 [202, 829]	676 [425, 849]	847 [847, 849]	813 [756, 848]	847 [847, 849]	821 [773, 849]
cheetah-run	894 [882, 904]	914 [911, 917]	740 [601, 861]	915 [913, 918]	758 [700, 811]	888 [845, 913]	909 [900, 918]	919 [918, 922]
dog-run	650 [610, 683]	569 [546, 595]	361 [222, 500]	232 [168, 293]	299 [248, 352]	632 [616, 650]	306 [205, 382]	591 [542, 638]
dog-stand	969 [959, 977]	967 [959, 975]	729 [588, 866]	443 [200, 680]	742 [643, 842]	939 [913, 961]	936 [917, 953]	974 [967, 983]
dog-trot	863 [807, 901]	877 [845, 898]	670 [573, 750]	490 [385, 582]	741 [655, 813]	905 [898, 913]	616 [559, 672]	871 [828, 899]
dog-walk	920 [882, 942]	916 [908, 924]	655 [444, 783]	652 [492, 790]	773 [715, 829]	917 [904, 928]	914 [902, 928]	938 [927, 948]
finger-spin	799 [728, 858]	937 [917, 958]	459 [220, 637]	983 [980, 988]	927 [848, 983]	983 [982, 986]	987 [986, 989]	900 [803, 972]
finger-turn.easy	803 [746, 866]	953 [928, 975]	602 [545, 659]	977 [968, 987]	954 [925, 979]	924 [884, 963]	951 [907, 982]	946 [916, 973]
finger-turn.hard	757 [708, 809]	950 [908, 974]	491 [383, 629]	877 [680, 984]	945 [917, 970]	919 [856, 974]	965 [946, 979]	959 [941, 972]
fish-swim	754 [706, 790]	792 [772, 811]	638 [578, 714]	737 [648, 815]	358 [283, 446]	731 [695, 768]	682 [617, 749]	817 [787, 840]
hopper-hop	267 [170, 359]	251 [201, 295]	224 [64, 384]	455 [387, 514]	352 [305, 411]	451 [364, 543]	456 [401, 511]	287 [250, 324]
hopper-stand	875 [771, 936]	951 [948, 955]	923 [916, 931]	922 [912, 933]	922 [892, 950]	934 [906, 956]	953 [948, 959]	871 [758, 950]
humanoid-run	292 [241, 339]	200 [169, 236]	104 [71, 128]	197 [160, 231]	153 [121, 185]	396 [355, 435]	262 [241, 283]	182 [169, 199]
humanoid-stand	915 [901, 929]	868 [823, 907]	111 [33, 217]	785 [716, 842]	705 [645, 770]	902 [878, 919]	787 [770, 808]	864 [813, 911]
humanoid-walk	833 [767, 876]	662 [609, 721]	371 [174, 568]	682 [628, 734]	528 [479, 580]	901 [878, 922]	774 [742, 810]	695 [638, 771]
pendulum-swingup	577 [353, 767]	748 [594, 830]	417 [97, 739]	816 [771, 866]	854 [838, 870]	852 [834, 868]	836 [823, 851]	858 [843, 872]
quadruped-run	941 [930, 951]	947 [940, 954]	908 [857, 943]	940 [923, 958]	534 [362, 692]	908 [896, 921]	937 [915, 951]	938 [934, 943]
quadruped-walk	935 [884, 964]	963 [959, 968]	958 [953, 963]	968 [951, 983]	669 [527, 815]	958 [955, 962]	967 [961, 974]	954 [947, 961]
reacher-easy	970 [951, 983]	983 [983, 985]	871 [821, 927]	981 [973, 990]	980 [976, 984]	970 [953, 984]	974 [955, 986]	972 [954, 984]
reacher-hard	881 [822, 931]	977 [975, 979]	826 [766, 929]	972 [965, 981]	957 [930, 977]	929 [883, 966]	976 [974, 980]	975 [973, 979]
walker-run	792 [725, 829]	793 [766, 816]	530 [313, 719]	809 [801, 820]	816 [803, 830]	858 [856, 861]	852 [849, 856]	820 [817, 823]
walker-stand	989 [988, 990]	988 [987, 990]	965 [925, 988]	987 [976, 996]	986 [982, 989]	994 [993, 996]	989 [989, 991]	989 [986, 993]
walker-walk	977 [975, 980]	978 [978, 980]	976 [975, 979]	982 [978, 987]	981 [979, 984]	984 [981, 987]	981 [980, 984]	978 [976, 981]
Mean	807 [793, 819]	835 [829, 842]	651 [626, 676]	781 [768, 796]	758 [748, 771]	860 [856, 866]	833 [827, 840]	836 [826, 845]
Median	870 [851, 886]	927 [914, 932]	663 [631, 737]	878 [842, 898]	832 [824, 845]	913 [903, 919]	925 [911, 935]	909 [882, 931]
IQM	862 [849, 878]	907 [901, 915]	672 [648, 731]	853 [832, 882]	824 [813, 838]	912 [905, 920]	908 [903, 914]	904 [893, 917]

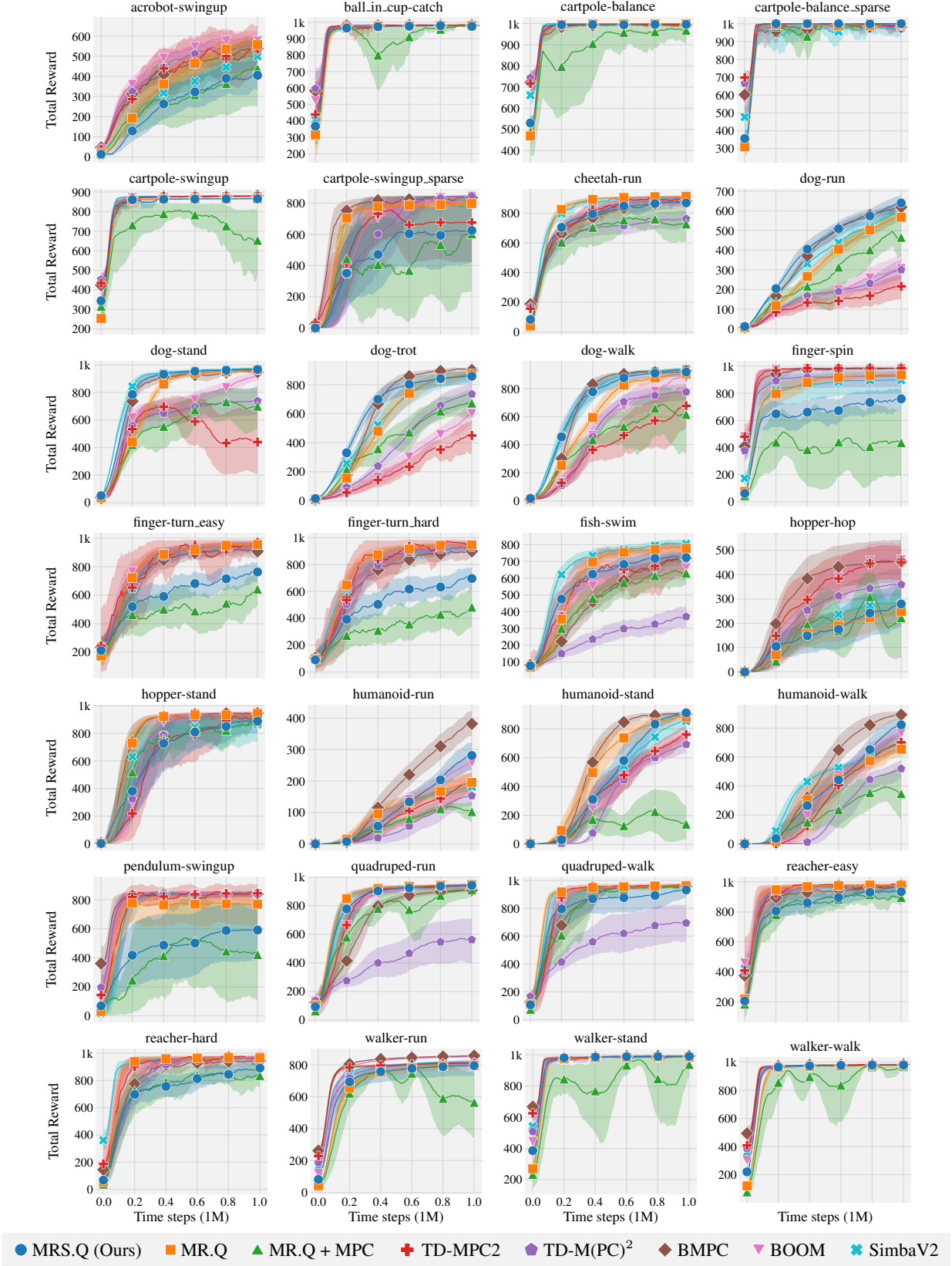


Figure 9. DMC learning curves. The shaded area captures a 95% stratified bootstrap confidence interval across 10 seeds.

C.3. HumanoidBench (No Hand)

Table 9. **HumanoidBench (No Hand) results.** Average final performance at 1M time steps. [Bracketed values] indicate 95% stratified bootstrap confidence intervals over 10 seeds. h1-reach results are not used in aggregate metrics.

Task	MRS.Q	MR.Q	MR.Q+MPC	TD-MPC2	TD-M(PC) ²	BMPC	BOOM	SimbaV2
h1-balance_hard	63 [61, 66]	69 [65, 76]	62 [59, 65]	56 [53, 60]	46 [44, 49]	59 [58, 61]	53 [52, 56]	136 [131, 143]
h1-balance_simple	475 [380, 557]	240 [208, 275]	214 [168, 266]	397 [373, 424]	121 [63, 206]	332 [246, 425]	373 [289, 435]	378 [255, 472]
h1-crawl	966 [961, 969]	931 [913, 949]	777 [767, 788]	968 [964, 972]	799 [737, 849]	795 [664, 932]	795 [664, 932]	872 [814, 932]
h1-hurdle	182 [165, 205]	128 [111, 146]	214 [184, 240]	387 [293, 483]	170 [155, 185]	204 [187, 222]	412 [337, 501]	61 [50, 74]
h1-maze	324 [319, 330]	345 [338, 353]	303 [287, 317]	232 [170, 291]	331 [331, 333]	291 [243, 319]	226 [168, 285]	335 [329, 343]
h1-pole	883 [856, 908]	602 [548, 653]	359 [284, 448]	730 [583, 845]	775 [693, 855]	575 [497, 651]	832 [761, 892]	269 [182, 374]
h1-reach	5091 [4875, 5346]	5991 [5486, 6442]	2531 [2227, 2811]	7753 [5514, 9314]	5753 [4809, 6762]	2706 [2415, 3078]	7440 [7313, 7516]	3645 [2440, 5403]
h1-run	791 [775, 802]	273 [198, 363]	651 [600, 702]	791 [789, 793]	444 [305, 589]	174 [143, 203]	765 [738, 788]	45 [31, 55]
h1-sit_hard	811 [779, 833]	512 [379, 654]	772 [749, 793]	799 [796, 803]	607 [459, 742]	361 [192, 503]	752 [703, 789]	716 [619, 814]
h1-sit_simple	863 [857, 872]	883 [860, 906]	811 [799, 821]	818 [818, 819]	865 [854, 878]	716 [636, 796]	814 [814, 815]	843 [824, 859]
h1-slide	453 [413, 500]	321 [295, 345]	245 [223, 266]	362 [304, 414]	533 [485, 583]	353 [291, 425]	385 [359, 421]	171 [148, 196]
h1-stair	199 [189, 214]	335 [262, 400]	139 [93, 197]	374 [216, 538]	227 [155, 332]	317 [253, 378]	379 [241, 514]	160 [76, 249]
h1-stand	837 [832, 843]	812 [795, 830]	807 [801, 813]	817 [815, 819]	848 [828, 867]	532 [445, 601]	811 [803, 817]	630 [518, 770]
h1-walk	819 [818, 821]	742 [636, 810]	682 [622, 744]	816 [815, 817]	807 [789, 818]	491 [302, 668]	814 [812, 817]	255 [195, 322]
Mean	590 [585, 595]	476 [456, 494]	464 [454, 479]	580 [560, 600]	506 [485, 529]	399 [372, 431]	551 [533, 571]	375 [360, 395]
Median	791 [773, 801]	345 [338, 400]	359 [298, 448]	730 [583, 793]	533 [470, 590]	353 [311, 418]	582 [551, 642]	269 [220, 333]
IQM	644 [634, 655]	447 [415, 475]	461 [444, 484]	613 [591, 648]	531 [497, 563]	382 [347, 409]	584 [563, 616]	314 [281, 353]

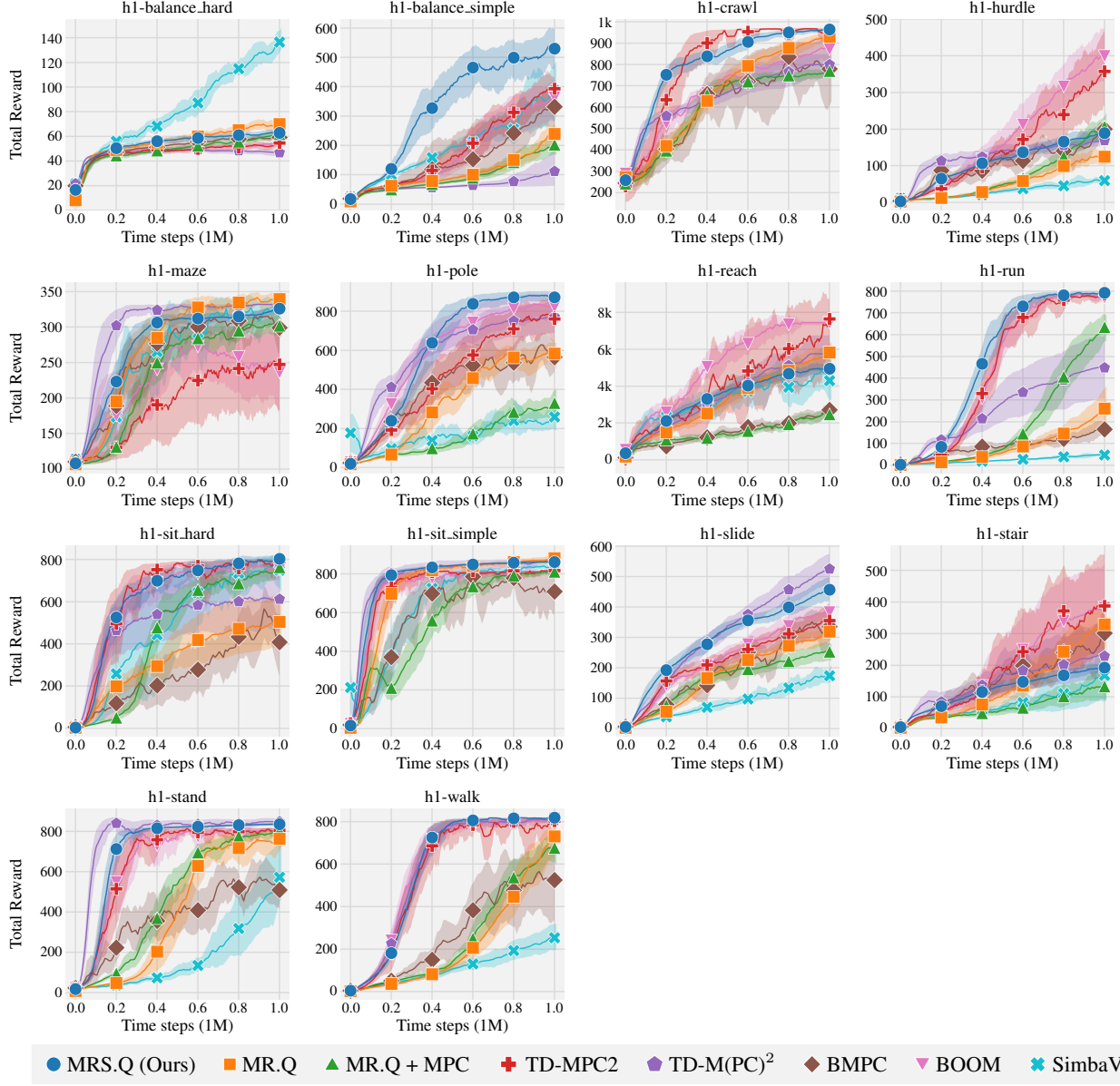


Figure 10. **HumanoidBench (Hand) learning curves.** The shaded area captures a 95% stratified bootstrap confidence interval across 10 seeds.

C.4. HumanoidBench (Hand)

Table 10. **HumanoidBench (Hand) results.** Average final performance at 1M time steps. [Bracketed values] indicate 95% stratified bootstrap confidence intervals over 10 seeds. `h1hand-reach` results are not used in aggregate metrics.

Task	MRS.Q	MR.Q	MR.Q+MPC	TD-MPC2	TD-M(PC) ²	BMPC	BOOM	SimbaV2
<code>h1hand-balance_hard</code>	80 [77, 84]	79 [76, 82]	73 [67, 80]	66 [59, 73]	64 [60, 70]	69 [67, 72]	69 [66, 73]	69 [61, 76]
<code>h1hand-balance_simple</code>	407 [311, 493]	143 [123, 162]	234 [190, 286]	93 [67, 125]	100 [85, 116]	278 [143, 423]	87 [78, 98]	93 [87, 101]
<code>h1hand-crawl</code>	898 [897, 901]	775 [734, 809]	680 [620, 724]	958 [933, 978]	958 [933, 978]	717 [574, 822]	717 [574, 822]	568 [539, 602]
<code>h1hand-hurdle</code>	128 [121, 135]	50 [35, 68]	103 [91, 115]	44 [25, 66]	179 [176, 184]	127 [92, 165]	50 [42, 58]	20 [16, 25]
<code>h1hand-maze</code>	294 [268, 312]	274 [245, 300]	250 [223, 277]	136 [111, 180]	136 [111, 180]	317 [300, 338]	284 [164, 355]	244 [174, 315]
<code>h1hand-pole</code>	833 [778, 890]	248 [221, 276]	236 [212, 262]	139 [103, 176]	821 [696, 930]	499 [385, 614]	184 [121, 256]	91 [79, 104]
<code>h1hand-reach</code>	2702 [2375, 3000]	3972 [3436, 4540]	3003 [2515, 3566]	6308 [4024, 8276]	6308 [4024, 8276]	2873 [2702, 3044]	2873 [2702, 3044]	3015 [2431, 3790]
<code>h1hand-run</code>	783 [699, 850]	44 [35, 55]	272 [236, 307]	65 [39, 97]	542 [394, 684]	109 [59, 160]	45 [28, 63]	19 [17, 24]
<code>h1hand-sit_hard</code>	864 [820, 904]	658 [543, 771]	785 [734, 834]	369 [118, 628]	748 [607, 857]	465 [325, 597]	421 [288, 571]	371 [170, 557]
<code>h1hand-sit_simple</code>	928 [927, 930]	780 [682, 876]	875 [849, 900]	177 [40, 395]	177 [40, 395]	502 [309, 689]	727 [482, 906]	671 [587, 757]
<code>h1hand-slide</code>	337 [307, 365]	179 [153, 206]	215 [200, 230]	104 [65, 146]	448 [417, 477]	331 [256, 391]	156 [109, 211]	58 [53, 64]
<code>h1hand-stair</code>	188 [152, 232]	156 [123, 191]	93 [80, 111]	59 [52, 68]	116 [100, 132]	230 [134, 347]	59 [51, 70]	60 [52, 68]
<code>h1hand-stand</code>	923 [905, 933]	454 [355, 562]	761 [668, 858]	296 [195, 394]	296 [195, 394]	620 [511, 727]	397 [206, 610]	62 [48, 76]
<code>h1hand-walk</code>	812 [691, 890]	165 [114, 237]	372 [294, 453]	308 [250, 365]	921 [909, 933]	705 [555, 832]	299 [189, 395]	48 [41, 59]
Mean	575 [569, 582]	308 [295, 324]	381 [372, 392]	216 [193, 245]	438 [414, 463]	375 [350, 404]	232 [201, 262]	183 [170, 196]
Median	783 [675, 814]	179 [159, 237]	250 [231, 278]	136 [86, 152]	448 [394, 476]	331 [310, 422]	170 [130, 209]	69 [63, 77]
IQM	619 [607, 632]	231 [214, 249]	323 [308, 338]	144 [114, 168]	406 [368, 441]	375 [341, 400]	180 [146, 207]	97 [84, 108]

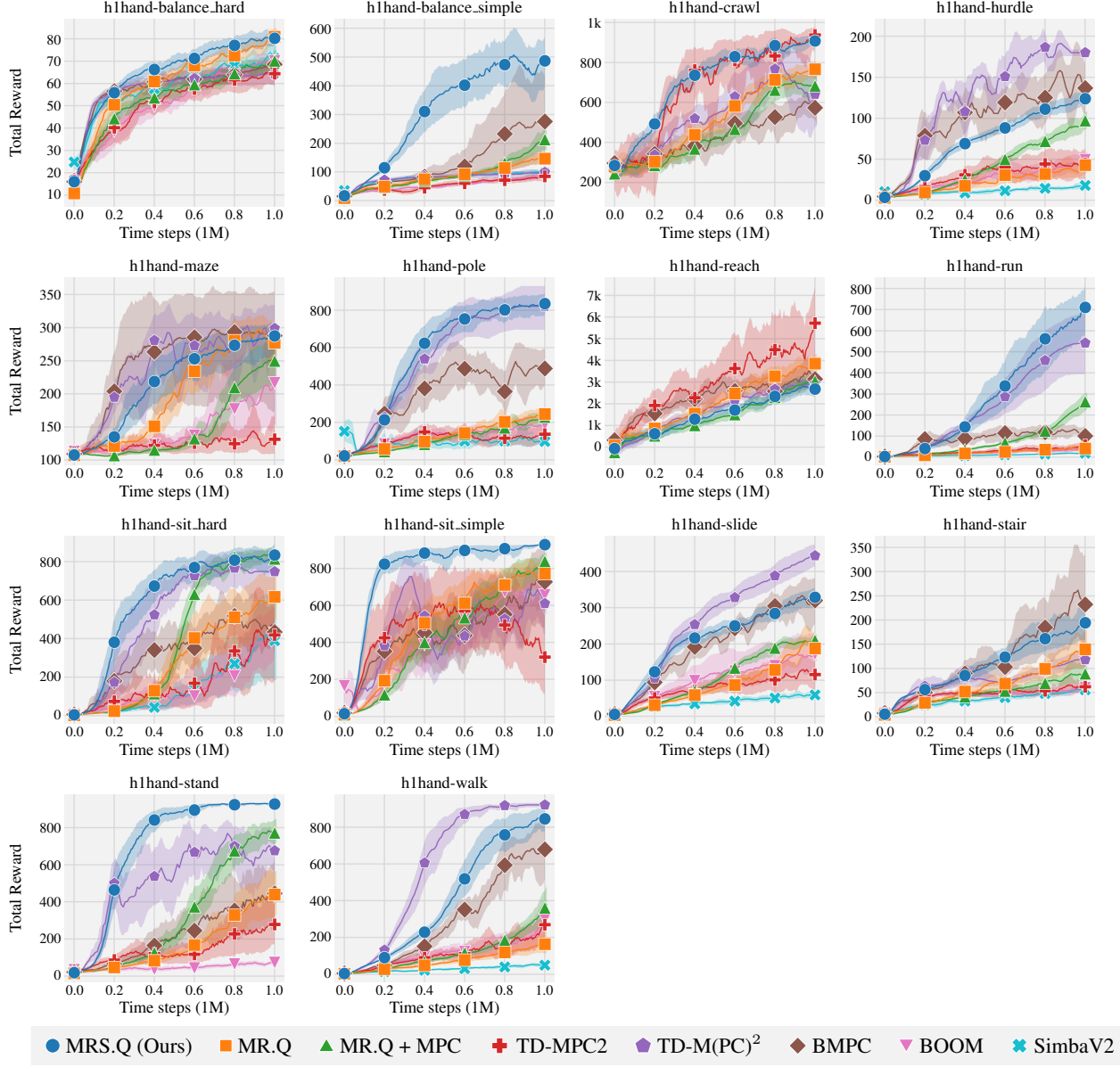


Figure 11. **HumanoidBench (No Hand) learning curves.** The shaded area captures a 95% stratified bootstrap confidence interval across 10 seeds.

D. Ablation Results

Table 11. Gym ablation results for MRS.Q. Average final performance at 1M time steps. [Bracketed values] indicate 95% stratified bootstrap confidence intervals over 5 seeds.

Task	MRS.Q	2 VF	5 VF	20 VF	Exploration	SEM	Min (Ensemble)	Min (MPC)
Ant	7189 [6394, 7804]	6275 [4468, 7622]	6121 [4723, 7267]	6260 [5578, 7140]	6822 [6265, 7283]	4743 [2739, 6749]	5789 [4324, 7134]	5682 [4282, 7091]
HalfCheetah	14649 [14149, 15158]	14778 [14454, 15158]	11574 [7195, 14705]	13751 [13477, 14209]	12438 [12155, 12724]	12791 [12254, 13329]	14355 [14066, 14659]	13137 [13138, 13138]
Hopper	3248 [2945, 3516]	1538 [1169, 1855]	2874 [2225, 3415]	3233 [2760, 3526]	2866 [1751, 3468]	3131 [2699, 3564]	1498 [1101, 1855]	1899 [961, 2969]
Humanoid	10526 [10186, 10816]	4425 [2571, 5506]	7366 [5766, 8769]	10486 [10206, 10742]	9933 [9292, 10513]	8912 [7617, 10208]	8418 [6622, 10214]	2730 [2158, 3461]
Walker2d	5699 [5304, 5981]	1028 [761, 1269]	3561 [2687, 4449]	5917 [5660, 6189]	5073 [3664, 5971]	4566 [1633, 7501]	1295 [969, 1563]	1251 [1100, 1404]
Mean	1.54 [1.46, 1.60]	0.91 [0.82, 1.01]	1.17 [1.06, 1.27]	1.49 [1.44, 1.54]	1.40 [1.29, 1.49]	1.26 [0.92, 1.59]	1.05 [0.99, 1.10]	0.82 [0.73, 0.91]
Median	1.44 [1.36, 1.52]	0.85 [0.49, 1.07]	1.09 [0.91, 1.36]	1.50 [1.41, 1.55]	1.29 [1.15, 1.51]	1.20 [0.84, 1.70]	1.35 [1.10, 1.37]	0.59 [0.43, 0.92]
IQM	1.54 [1.45, 1.62]	0.90 [0.76, 0.99]	1.14 [0.99, 1.27]	1.46 [1.39, 1.51]	1.39 [1.29, 1.49]	1.19 [0.90, 1.62]	1.09 [0.94, 1.16]	0.78 [0.67, 0.90]

Table 12. Gym ablation results for TD-MPC2. Average final performance at 1M time steps. [Bracketed values] indicate 95% stratified bootstrap confidence intervals over 10 seeds.

Task	TD-MPC2	Min (10)
Ant	1938 [795, 3694]	4412 [3008, 5963]
HalfCheetah	11188 [6871, 15475]	9024 [2649, 15400]
Hopper	267 [170, 360]	237 [193, 265]
Humanoid	297 [260, 336]	319 [218, 418]
Walker2d	1537 [668, 2592]	1236 [888, 1629]
Mean	0.41 [0.27, 0.57]	0.48 [0.33, 0.62]
Median	0.39 [0.17, 0.65]	0.31 [0.22, 0.40]
IQM	0.32 [0.15, 0.53]	0.41 [0.21, 0.54]

Table 13. DMC ablation results for MRS.Q. Average final performance at 1M time steps. [Bracketed values] indicate 95% stratified bootstrap confidence intervals over 5 seeds.

Task	MRS.Q	2 VF	5 VF	20 VF	Exploration	SEM	Min (Ensemble)	Min (MPC)
acrobot-swingup	468 [417, 515]	604 [561, 646]	516 [438, 583]	304 [231, 378]	429 [307, 520]	496 [311, 658]	647 [605, 691]	644 [614, 674]
ball_in_cup-catch	979 [977, 982]	974 [973, 976]	979 [977, 982]	965 [943, 978]	977 [976, 978]	975 [975, 977]	976 [975, 977]	976 [975, 978]
cartpole-balance	995 [992, 998]	996 [993, 998]	997 [997, 999]	995 [993, 998]	998 [997, 999]	997 [995, 999]	996 [995, 998]	997 [996, 998]
cartpole-balance_sparse	1000 [1000, 1000]	1000 [1000, 1000]	1000 [1000, 1000]	1000 [1000, 1000]	1000 [1000, 1000]	1000 [1000, 1000]	1000 [1000, 1000]	1000 [1000, 1000]
cartpole-swingup	866 [865, 867]	864 [862, 866]	864 [862, 866]	863 [858, 867]	866 [866, 867]	867 [867, 867]	696 [347, 876]	866 [866, 867]
cartpole-swingup_sparse	623 [424, 788]	794 [795, 795]	745 [633, 807]	754 [754, 754]	725 [726, 726]	808 [808, 808]	815 [815, 815]	808 [808, 808]
cheetah-run	894 [882, 904]	868 [832, 896]	803 [707, 879]	872 [833, 898]	870 [857, 886]	858 [815, 901]	850 [816, 884]	714 [594, 814]
dog-run	650 [610, 683]	604 [578, 646]	659 [641, 678]	625 [552, 684]	626 [588, 664]	572 [471, 663]	663 [641, 689]	626 [592, 654]
dog-stand	969 [959, 977]	906 [883, 932]	953 [928, 973]	946 [898, 975]	974 [972, 978]	966 [952, 980]	885 [874, 901]	896 [887, 905]
dog-trot	863 [807, 901]	731 [644, 822]	838 [786, 889]	899 [886, 913]	843 [748, 898]	726 [727, 727]	890 [876, 906]	892 [874, 907]
dog-walk	920 [882, 942]	821 [786, 849]	924 [913, 934]	927 [917, 939]	929 [922, 938]	933 [929, 936]	895 [877, 913]	849 [840, 864]
finger-spin	799 [728, 858]	813 [764, 863]	857 [793, 917]	710 [628, 776]	931 [893, 961]	603 [484, 723]	854 [696, 980]	956 [911, 982]
finger-turn_easy	803 [746, 866]	661 [503, 820]	751 [701, 804]	772 [634, 915]	784 [725, 843]	602 [512, 693]	815 [726, 905]	687 [486, 836]
finger-turn_hard	757 [708, 809]	555 [492, 626]	692 [639, 738]	723 [632, 813]	603 [522, 718]	548 [484, 672]	767 [615, 919]	708 [533, 871]
fish-swim	754 [706, 790]	707 [675, 741]	729 [688, 765]	739 [701, 780]	744 [707, 779]	714 [651, 772]	752 [722, 779]	761 [744, 782]
hopper-hop	267 [170, 359]	379 [335, 450]	362 [291, 437]	149 [28, 270]	136 [53, 224]	377 [147, 612]	438 [334, 559]	405 [340, 498]
hopper-stand	875 [771, 936]	909 [869, 949]	926 [904, 947]	599 [285, 910]	636 [324, 909]	474 [2, 948]	929 [889, 953]	923 [886, 950]
humanoid-run	292 [241, 339]	151 [124, 183]	308 [272, 342]	215 [164, 286]	257 [230, 285]	243 [184, 320]	285 [233, 337]	277 [209, 346]
humanoid-stand	915 [901, 929]	464 [402, 528]	814 [755, 877]	915 [900, 930]	858 [780, 911]	850 [826, 871]	730 [619, 831]	733 [652, 791]
humanoid-walk	833 [767, 876]	527 [466, 583]	818 [751, 874]	811 [699, 877]	747 [604, 857]	769 [769, 769]	715 [624, 802]	726 [677, 784]
pendulum-swingup	577 [353, 767]	829 [813, 846]	605 [366, 835]	424 [0, 849]	422 [0, 845]	820 [820, 820]	836 [823, 849]	832 [816, 849]
quadruped-run	941 [930, 951]	949 [945, 953]	946 [942, 951]	948 [945, 952]	949 [945, 955]	932 [900, 953]	949 [938, 957]	947 [939, 954]
quadruped-walk	935 [884, 964]	959 [953, 967]	963 [960, 966]	965 [959, 972]	969 [962, 975]	976 [971, 980]	968 [962, 974]	967 [961, 974]
reacher-easy	970 [951, 983]	964 [925, 985]	906 [851, 958]	980 [976, 985]	943 [903, 983]	982 [980, 984]	962 [923, 986]	903 [745, 984]
reacher-hard	881 [822, 931]	664 [490, 822]	836 [779, 895]	827 [722, 929]	935 [859, 978]	679 [679, 679]	876 [873, 880]	867 [798, 937]
walker-run	792 [725, 829]	829 [826, 836]	827 [825, 831]	828 [827, 830]	811 [808, 816]	823 [821, 825]	830 [824, 837]	835 [830, 841]
walker-stand	989 [988, 990]	990 [989, 993]	988 [986, 991]	992 [990, 994]	990 [988, 993]	991 [987, 995]	990 [988, 993]	992 [990, 994]
walker-walk	977 [975, 980]	980 [978, 983]	978 [975, 981]	980 [978, 982]	980 [978, 984]	982 [978, 986]	982 [980, 985]	981 [980, 984]
Mean	807 [793, 819]	768 [756, 780]	807 [794, 819]	776 [751, 799]	783 [763, 806]	770 [745, 796]	821 [807, 836]	813 [801, 825]
Median	870 [851, 886]	825 [808, 840]	837 [827, 865]	845 [838, 887]	862 [856, 876]	821 [818, 845]	852 [840, 883]	858 [835, 864]
IQM	862 [849, 878]	810 [800, 826]	847 [839, 862]	845 [830, 872]	853 [849, 868]	810 [807, 843]	853 [839, 877]	846 [837, 866]

Table 14. DMC ablation results for TD-MPC2. Average final performance at 1M time steps. [Bracketed values] indicate 95% stratified bootstrap confidence intervals over 10 seeds.

Task	TD-MPC2	Min (10)
acrobot-swingup	554 [391, 690]	489 [281, 633]
ball_in_cup-catch	979 [973, 986]	984 [982, 986]
cartpole-balance	998 [998, 999]	998 [996, 1000]
cartpole-balance_sparse	970 [912, 1000]	1000 [1000, 1000]
cartpole-swingup	879 [875, 882]	877 [871, 882]
cartpole-swingup_sparse	676 [425, 849]	850 [850, 850]
cheetah-run	915 [913, 918]	886 [864, 908]
dog-run	232 [168, 293]	327 [299, 355]
dog-stand	443 [200, 680]	813 [533, 958]
dog-trot	490 [385, 582]	590 [538, 643]
dog-walk	652 [492, 790]	729 [383, 916]
finger-spin	983 [980, 988]	984 [981, 987]
finger-turn_easy	977 [968, 987]	963 [924, 986]
finger-turn_hard	877 [680, 984]	978 [972, 982]
fish-swim	737 [648, 815]	713 [660, 774]
hopper-hop	455 [387, 514]	394 [326, 476]
hopper-stand	922 [912, 933]	922 [889, 942]
humanoid-run	197 [160, 231]	250 [216, 289]
humanoid-stand	785 [716, 842]	767 [693, 837]
humanoid-walk	682 [628, 734]	815 [805, 827]
pendulum-swingup	816 [771, 866]	781 [641, 878]
quadruped-run	940 [923, 958]	939 [934, 943]
quadruped-walk	968 [951, 983]	967 [965, 970]
reacher-easy	981 [973, 990]	979 [968, 985]
reacher-hard	972 [965, 981]	969 [967, 973]
walker-run	809 [801, 820]	809 [801, 821]
walker-stand	987 [976, 996]	993 [992, 994]
walker-walk	982 [978, 987]	983 [983, 984]
Mean	781 [768, 796]	813 [797, 827]
Median	878 [842, 898]	882 [871, 919]
IQM	853 [832, 882]	881 [867, 906]

Table 15. **HumanoidBench (No Hand) ablation results for MRS.Q**. Average final performance at 1M time steps. [Bracketed values] indicate 95% stratified bootstrap confidence intervals over 5 seeds. h1-reach results are not used in aggregate metrics.

Task	MRS.Q	2 VF	5 VF	20 VF	Exploration	SEM	Min (Ensemble)	Min (MPC)
h1-balance_hard	63 [61, 66]	58 [52, 65]	64 [61, 68]	65 [56, 75]	56 [56, 58]	67 [63, 72]	61 [55, 67]	61 [57, 67]
h1-balance_simple	475 [380, 557]	278 [218, 338]	532 [470, 608]	569 [493, 653]	303 [264, 346]	512 [361, 600]	430 [344, 517]	491 [393, 558]
h1-crawl	966 [961, 969]	841 [760, 914]	954 [948, 960]	966 [961, 971]	968 [965, 972]	971 [970, 973]	948 [928, 969]	949 [921, 964]
h1-hurdle	182 [165, 205]	86 [75, 100]	177 [156, 199]	205 [193, 220]	185 [173, 200]	414 [415, 415]	122 [104, 140]	135 [116, 155]
h1-maze	324 [319, 330]	297 [283, 316]	320 [309, 331]	322 [313, 332]	319 [316, 325]	356 [356, 356]	318 [315, 324]	312 [297, 324]
h1-pole	883 [856, 908]	646 [453, 818]	832 [760, 885]	908 [907, 909]	842 [719, 906]	909 [909, 910]	868 [830, 906]	824 [751, 898]
h1-reach	5091 [4875, 5346]	3784 [3473, 4081]	4774 [4521, 5026]	4812 [4566, 5073]	5597 [5103, 5850]	6091 [5612, 6570]	5168 [5167, 5169]	4838 [4550, 5230]
h1-run	791 [775, 802]	277 [202, 382]	736 [673, 783]	801 [797, 806]	794 [792, 796]	804 [804, 805]	575 [499, 651]	668 [615, 712]
h1-sit_hard	811 [779, 833]	539 [371, 683]	779 [751, 807]	786 [754, 818]	735 [702, 773]	786 [743, 829]	820 [809, 839]	767 [720, 815]
h1-sit_simple	863 [857, 872]	815 [812, 818]	848 [832, 859]	865 [858, 874]	842 [838, 847]	864 [864, 864]	854 [848, 857]	859 [855, 863]
h1-slide	453 [413, 500]	213 [179, 249]	328 [292, 362]	588 [481, 696]	408 [355, 471]	808 [797, 820]	315 [279, 353]	320 [294, 352]
h1-stair	199 [189, 214]	107 [84, 140]	208 [165, 281]	262 [172, 418]	263 [163, 445]	240 [220, 255]	140 [119, 172]	150 [110, 221]
h1-stand	837 [832, 843]	751 [693, 809]	824 [822, 827]	856 [847, 869]	823 [822, 824]	857 [853, 861]	819 [818, 820]	818 [816, 822]
h1-walk	819 [818, 821]	394 [273, 528]	784 [729, 817]	823 [820, 826]	817 [816, 818]	830 [824, 837]	712 [613, 801]	791 [730, 829]
Mean	590 [585, 595]	408 [390, 429]	568 [560, 577]	617 [605, 631]	566 [558, 578]	647 [638, 657]	537 [520, 553]	550 [537, 563]
Median	791 [773, 801]	297 [281, 367]	736 [673, 783]	786 [754, 803]	735 [702, 759]	804 [797, 820]	575 [499, 650]	668 [615, 712]
IQM	644 [634, 655]	378 [352, 407]	615 [599, 629]	678 [666, 696]	600 [586, 620]	716 [697, 735]	570 [545, 592]	596 [573, 614]

Table 16. **HumanoidBench (No Hand) results for TD-MPC2**. Average final performance at 1M time steps. [Bracketed values] indicate 95% stratified bootstrap confidence intervals over 10 seeds. h1-reach results are not used in aggregate metrics.

Task	TD-MPC2	Min (10)
h1-balance_hard	56 [53, 60]	52 [51, 54]
h1-balance_simple	397 [373, 424]	353 [321, 394]
h1-crawl	968 [964, 972]	969 [968, 971]
h1-hurdle	387 [293, 483]	365 [308, 428]
h1-maze	232 [170, 291]	216 [157, 261]
h1-pole	730 [583, 845]	851 [762, 901]
h1-reach	7753 [5514, 9314]	6943 [6319, 7686]
h1-run	791 [789, 793]	792 [790, 794]
h1-sit_hard	799 [796, 803]	801 [801, 802]
h1-sit_simple	818 [818, 819]	818 [818, 820]
h1-slide	362 [304, 414]	400 [355, 452]
h1-stair	374 [216, 538]	516 [388, 615]
h1-stand	817 [815, 819]	818 [818, 820]
h1-walk	816 [815, 817]	816 [815, 818]
Mean	580 [560, 600]	598 [583, 612]
Median	730 [583, 793]	792 [762, 794]
IQM	613 [591, 648]	644 [625, 666]

Table 17. **HumanoidBench (Hand) ablation results for MRS.Q.** Average final performance at 1M time steps. [Bracketed values] indicate 95% stratified bootstrap confidence intervals over 5 seeds. `h1hand-reach` results are not used in aggregate metrics.

Task	MRS.Q	2 VF	5 VF	20 VF	Exploration	SEM	Min (Ensemble)	Min (MPC)
<code>h1hand-balance_hard</code>	80 [77, 84]	70 [65, 77]	83 [77, 90]	79 [75, 85]	82 [80, 85]	76 [75, 77]	83 [78, 88]	81 [77, 88]
<code>h1hand-balance_simple</code>	407 [311, 493]	107 [90, 134]	344 [256, 440]	465 [343, 608]	224 [160, 306]	635 [635, 635]	309 [249, 374]	330 [242, 423]
<code>h1hand-crawl</code>	898 [897, 901]	595 [510, 691]	788 [773, 806]	860 [817, 911]	813 [774, 853]	969 [969, 969]	784 [737, 836]	756 [692, 835]
<code>h1hand-hurdle</code>	128 [121, 135]	39 [31, 49]	81 [74, 88]	158 [140, 177]	116 [107, 127]	244 [225, 270]	75 [64, 84]	75 [65, 84]
<code>h1hand-maze</code>	294 [268, 312]	231 [188, 270]	283 [262, 303]	250 [198, 304]	288 [269, 308]	315 [310, 318]	271 [228, 299]	280 [254, 306]
<code>h1hand-pole</code>	833 [778, 890]	152 [130, 188]	625 [544, 699]	938 [889, 965]	888 [864, 913]	921 [876, 966]	357 [242, 499]	341 [262, 420]
<code>h1hand-reach</code>	2702 [2375, 3000]	1688 [1587, 1833]	2106 [1883, 2365]	3441 [3065, 3889]	2916 [2683, 3150]	4096 [3780, 4412]	2263 [2264, 2264]	2683 [2343, 3025]
<code>h1hand-run</code>	783 [699, 850]	30 [22, 39]	243 [197, 293]	823 [726, 892]	611 [472, 735]	915 [914, 919]	171 [99, 243]	142 [113, 168]
<code>h1hand-sit_hard</code>	864 [820, 904]	185 [99, 273]	726 [667, 782]	854 [817, 892]	908 [904, 912]	814 [747, 883]	643 [505, 800]	629 [445, 746]
<code>h1hand-sit_simple</code>	928 [927, 930]	273 [189, 362]	885 [858, 909]	930 [929, 932]	905 [860, 928]	928 [928, 930]	918 [910, 926]	894 [864, 924]
<code>h1hand-slide</code>	337 [307, 365]	131 [115, 147]	243 [226, 264]	396 [359, 434]	314 [265, 363]	880 [849, 913]	222 [188, 250]	170 [153, 186]
<code>h1hand-stair</code>	188 [152, 232]	57 [52, 62]	148 [113, 189]	275 [183, 428]	287 [166, 396]	234 [235, 235]	126 [115, 137]	141 [108, 175]
<code>h1hand-stand</code>	923 [905, 933]	283 [183, 411]	872 [836, 905]	936 [936, 937]	930 [928, 933]	934 [934, 935]	789 [681, 866]	825 [769, 877]
<code>h1hand-walk</code>	812 [691, 890]	129 [112, 147]	468 [393, 535]	890 [827, 934]	795 [719, 892]	933 [933, 934]	249 [174, 329]	296 [224, 369]
Mean	575 [569, 582]	176 [164, 187]	445 [432, 458]	604 [593, 619]	551 [536, 569]	677 [675, 679]	384 [372, 398]	381 [370, 392]
Median	783 [675, 814]	131 [122, 147]	344 [281, 420]	823 [726, 859]	611 [472, 735]	880 [876, 913]	271 [239, 312]	296 [258, 322]
IQM	619 [607, 632]	144 [128, 155]	419 [397, 440]	652 [641, 675]	571 [546, 602]	773 [769, 774]	317 [301, 334]	312 [290, 336]

Table 18. **HumanoidBench (Hand) results for TD-MPC2.** Average final performance at 1M time steps. [Bracketed values] indicate 95% stratified bootstrap confidence intervals over 10 seeds. `h1hand-reach` results are not used in aggregate metrics.

Task	TD-MPC2	Min (10)
<code>h1hand-balance_hard</code>	66 [59, 73]	65 [64, 68]
<code>h1hand-balance_simple</code>	93 [67, 125]	83 [76, 93]
<code>h1hand-crawl</code>	958 [933, 978]	903 [862, 946]
<code>h1hand-hurdle</code>	44 [25, 66]	42 [12, 75]
<code>h1hand-maze</code>	136 [111, 180]	142 [123, 160]
<code>h1hand-pole</code>	139 [103, 176]	235 [191, 320]
<code>h1hand-reach</code>	6308 [4024, 8276]	5338 [4781, 5896]
<code>h1hand-run</code>	65 [39, 97]	84 [50, 133]
<code>h1hand-sit_hard</code>	369 [118, 628]	637 [332, 852]
<code>h1hand-sit_simple</code>	177 [40, 395]	647 [267, 921]
<code>h1hand-slide</code>	104 [65, 146]	196 [171, 220]
<code>h1hand-stair</code>	59 [52, 68]	69 [62, 79]
<code>h1hand-stand</code>	296 [195, 394]	623 [495, 752]
<code>h1hand-walk</code>	308 [250, 365]	554 [326, 783]
Mean	216 [193, 245]	329 [287, 363]
Median	136 [86, 152]	196 [171, 216]
IQM	144 [114, 168]	274 [206, 286]